

Survey paper

Beyond immutability: A comprehensive review of redactable blockchain systems

Imane El Abid ^{a,*}, Karim Boubouh ^b, Yahya Benkaouz ^{a,c}^a College of Computing, Mohammed VI Polytechnic University, Benguerir, Morocco^b 6G Research Center Khalifa University, Abu Dhabi, United Arab Emirates^c Conception and Systems Laboratory, Faculty of Sciences, Mohammed V University in Rabat, Morocco

ARTICLE INFO

Keywords:

Blockchain
Immutability
Redactable blockchain
Chameleon hash
Privacy

ABSTRACT

With the exponential rise of the data-driven economy, data sharing has become a fundamental driver of communication and collaboration across a wide range of applications. Blockchain technology, in particular, has gained immense popularity as a tool for efficient data sharing and management far beyond cryptocurrencies. The rapid adoption of blockchain is attributed to the inherent security, tamper-proof nature, and immutability of the system, among other benefits. However, the strict immutability of conventional blockchain systems introduces limitations in adaptability and privacy for use cases requiring dynamic, privacy-centric data control. Specifically, certain application-level requirements, such as low tolerance for software errors, frequent updates, or high privacy standards, may necessitate control and update after publication on the blockchain. Blockchain redaction addresses these challenges by enabling controlled modifications to blockchain content while preserving structural and security guarantees. Although it is a relatively nascent field, research in blockchain redaction has started to build a substantial knowledge base in recent years. This survey provides a comprehensive, up-to-date analysis of state-of-the-art redaction techniques, systematically evaluating their security guarantees, performance, and design trade-offs. We propose a revised taxonomy of redaction methodologies, comparing them both analytically and experimentally, and highlighting their mechanisms, use cases, and limitations. Through this analysis, we showcase the potential of mutable blockchain designs to meet the evolving demands of modern applications without compromising core guarantees such as verifiability, auditability, and integrity.

1. Introduction

Blockchain is a distributed protocol that allows a peer-to-peer network to maintain an immutable ledger. This ledger consists of a linear sequence of blocks, continuously verified and updated through cryptographic primitives and distributed consensus algorithms [1]. Basically, the blockchain offers major attributes rendering transactions resistant to alteration, namely decentralization, auditability, and data immutability. Blockchain immutability stems from the computational unfeasibility to go back to the records history and make a change retroactively without altering all subsequent blocks. Henceforth, a direct consequence of an immutable blockchain is an indisputable history and permanent data records, which align seamlessly with the requirements of cryptocurrency systems.

Renowned cryptocurrency systems, such as Bitcoin [2] and Ethereum [3], offer a high degree of traceability and immutability due to their

reliance on a fully decentralized network of nodes to validate and record transactions. Every transaction within the system is subject to stringent validation through consensus protocols such as Proof-of-Work (PoW) [2] or Proof-of-Stake (PoS) [3]. Once a transaction is approved and recorded on the blockchain, it becomes immutable and unalterable without the agreement of the entire network. This core immutability property guarantees a verifiable and permanent record, while preventing fraudulent activities such as double-spending that is commonly associated with digital payment systems [4].

Blockchain has supported privacy-preserving applications [5] and verifiable data deletion in external platforms [6], yet remains inherently non-rectifiable. The inherent immutability of blockchain also triggers critical questions about data privacy on the distributed ledger. One concern is that the immutability of the data on the blockchain means that any errors or mistakes made during transactions submission are irrevocable and cannot be corrected or deleted. Therefore, any inadvertent

* Corresponding author.

E-mail address: imane.elabid@um6p.ma (I. El Abid).<https://doi.org/10.1016/j.comnet.2026.112133>

Received 3 October 2025; Received in revised form 13 January 2026; Accepted 17 February 2026

Available online 25 February 2026

1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1
Comparison of our survey with other existing surveys.

Surveys	[7]	[8]	[9]	[10]	[11]	[12]	Our work
Presents benefits and limitations of redaction methods	~	✓	✓	✓	~	x	✓
Provides an updated and detailed analysis of redaction methods	x	x	x	✓	✓	x	✓
Covers subsequent improvements	x	~	~	✓	~	x	✓
Include details about recent methods	x	x	x	x	x	x	✓
Discusses open issues	x	x	x	✓	✓	~	✓
Provides experimental benchmark	x	x	x	x	x	x	✓

x: Not addressed. ~: Partially addressed. ✓: Addressed.

or intentional inclusion of sensitive or personal data in the ledger induces the permanent storage and accessibility of such data to the entire network.

When personal data is involved, immutability admittedly conflicts with numerous privacy regulations and data protection rights. One notable example is the conflict between blockchain immutability and the “*right to be forgotten*” enshrined in regulations like the European General Data Protection Regulation (GDPR) [13]. The transparency and visibility of data on the blockchain to all participants in the network pose a potential risk of unauthorized access to sensitive or personal information. In some cases, certain blockchain applications accidentally or intentionally expose sensitive data, making it permanently and publicly accessible to the entire network [14]. Examples of such data include healthcare and insurance records, personal addresses, and access credentials. These applications may not adequately consider the privacy implications and fail to implement appropriate measures to protect sensitive data [15].

Furthermore, logical errors and exploitable vulnerabilities in programmed smart contracts can compromise the entire system. An example of software flaws is the *Ethereum DAO attack*, which occurred in June 2016 and resulted in attackers draining more than 50 million dollars from the Ethereum system [16]. To resolve the loss of funds for their users, Ethereum community executed a hard fork. The hard fork necessitates modifications to the blockchain code to reverse the transactions and mandate rewriting all blocks after the alteration point. As a result, the Ethereum community created a new blockchain that split from the original chain, resulting in two distinct blockchains.

Apart from hard fork, a trivial solution to tackle personal data privacy in blockchain involves storing personal data off-chain, using a centralized server or cloud-based storage system. In this case, only the hash of the data will be stored in the blockchain ledger [17–19]. This approach, however, abandons the principle of decentralization and restricts the use of blockchain to a mere verification tool or a lookup table.

An alternative solution is to encrypt all personal data stored on the blockchain, with the option to destroy the decryption key if the data needs to be erased [18]. Although this approach provides a level of privacy, it also hinders the traceability of the data and limits access to data history for multiple parties, and thus may impede the ability of various parties to verify the integrity and authenticity of the data stored on the blockchain.

Rather than bypassing immutability, a creative approach promoting *mutable* blockchain [20] has emerged. The idea of a redactable blockchain as a type of blockchain that allows users to edit the data in previously confirmed blocks seems controversial, as the blockchain is by definition an immutable structure. In recent years, this new paradigm gained increasing interest in the blockchain community, and several follow-up studies have been conducted to investigate its potential use and limitations.

In this work, we investigate disruptive redaction methods that challenge conventional blockchain immutability. Through a comprehensive analytical and experimental evaluation, we establish a benchmark for the state-of-the-art blockchain redaction techniques. A deeper understanding of these disruptive techniques aims to create advanced blockchain models that exhibit high efficiency. To contextualize our

contribution within the established literature, we compare our work to prior surveys on redactable blockchain systems. While the number of existing surveys remains limited [7–12], most have focused on a narrow subset of redaction approaches. Primarily, our study presents a broader and more systematic exploration of the redaction paradigm, including a qualitative comparison of diverse solution classes. Critically, prior surveys lack empirical validation, in contrast to what we propose in our work.

Table 1 outlines the comparative scope of existing surveys against our work. Ultimately, the key contributions of this work may be succinctly summarized as follows:

- We define a unified taxonomy of redaction techniques grounded in the core pillars of blockchain immutability: *cryptographic*, *meta-transaction*, and *consensus-based* approaches. This classification reflects how each redaction technique necessarily disrupts one or more fundamental pillar of immutability, specifically, cryptographic binding, transaction semantics, and consensus. We further decompose each main class into subclasses to trace the technical evolution and operational trade-offs.
- We are the first to propose an experimental benchmark to evaluate redaction techniques, quantifying their efficiency, performance overhead, and practical viability. Moreover, our open source implementation serves as a reusable foundation for future research, facilitating the inclusion and evaluation of new systems to enhance the efficacy of redactable blockchain designs.
- We identify and discuss several open issues and research directions to further explore the use of blockchain redaction. In particular, we examine issues related to redaction latency and scalability, discuss adoption barriers, and emphasize the critical importance of content scrutiny and post-redaction state consistency.

The remainder of the paper is structured as follows: Section 2 provides the essential background and foundations of blockchain technology, examining its architecture, immutability mechanisms, motivations for redaction, and functional requirements. In Section 3, we present our novel taxonomy of redaction techniques with a comprehensive categorization framework and detailed analysis of cryptographic, meta-transactions, and consensus-based approaches. Section 4 delivers our empirical evaluation and benchmarking results of the main redaction methods. Section 5 explores open problems and future research directions in blockchain redaction. Finally, we present our conclusions in Section 6.

2. Background & foundations

The foundational promise of blockchain, namely its immutability, is both its greatest strength and a significant obstacle to regulatory compliance and operational flexibility. This section introduces the foundations of blockchain immutability based on Bitcoin’s architecture as a reference. We then explore the key motivations driving redaction research, including regulatory compliance, ethical concerns over illicit content, and system-level optimization.

2.1. Blockchain architecture primer

Blockchain is a decentralized, distributed ledger technology where a peer-to-peer network of nodes collaborates to maintain a tamper-resistant record of transactions. These transactions can represent simple asset transfers like cryptocurrencies [2] or execute programmable smart contracts [21].

Each transaction is cryptographically signed with a user's private key, ensuring non-repudiation and integrity. Validated transactions are then batched into linked blocks, each block contains a cryptographic hash of the previous block, a Merkle tree aggregating all transactions, and metadata such as a nonce. This chaining via hashes creates an append-only ledger that forms the blockchain core structure. To formalize this architecture, we adopt the model defined in Garay et al. [22]. For ease of understanding, Fig. 1 depicts a diagrammatic presentation of the blockchain structure. A blockchain C_n can be represented as a linear sequence of n blocks. Each block B_i is defined by the tuple:

$$B_i = \langle s_i, x_i, ctr_i \rangle, \text{ with } i \in \{0, n\} \quad (1)$$

We refer to the hash of the previous block by $s_i \in \{0, 1\}^\kappa$ where the hash size is κ bits. $x_i \in \{0, 1\}^*$ is the binary sequence that represents the data and $ctr_i \in \mathbb{N}$ is the *nonce*. The blocks B_0 and B_n are respectively the *genesis block* and the latest block. We note that for the genesis block B_0 there is no previous block to reference. Therefore, B_0 's components (i.e., ctr_{-1} and s_{-1}) are typically hard-coded directly into the blockchain.

When a block B_{n+1} is added to the chain, it is recorded as $C_{n+1} = C_n \parallel B_{n+1}$. The block-to-block hash link is expressed by $s_{n+1} = H(ctr_n, G(s_n, x_n))$ where $H : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ and $G : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ be a collision-resistant cryptographic hash functions. $G(s_i, x_i)$ refers to the Merkle root of the transactions. A block is considered valid if it meets certain requirements set by the application. In permissionless systems like Bitcoin, consensus is achieved through Proof-of-Work (PoW), where miners compete to solve computational puzzles by finding a nonce such that the resulting block hash is below a global difficulty threshold [2]. These puzzles are hard to solve but easy to verify, ensuring the integrity of the blockchain and resistance to tampering, even in the presence of bounded adversaries [23]. Formally, a block B_i is considered valid if and only if the following equation holds:

$$\text{ValidBlock}_q^D(B_i) := (H(ctr_i, G(s_i, x_i)) < D) \wedge (ctr_i < q) = 1 \quad (2)$$

with D being the current block difficulty level, and q is the maximum allowed number of hash queries in a round.

More energy-efficient consensus protocols such as Proof-of-Stake (PoS) [3] have emerged to address PoW's limitations. PoS replaces computational effort with economic commitment by selecting block proposers based on the amount of stake they hold in the network. Alternatively, consensusless protocols like Snowflake [24], LightX [25] and AT2 [26] use randomized mechanisms to achieve fast convergence and reduce coordination overhead.

When a miner finds a valid nonce that produces a valid block hash, the miner broadcasts the new block for peer verification. Peers verify both the cryptographic signatures and that the block hash meets the protocol-specific difficulty criteria. Upon acceptance, the block is appended, and the miner earns a reward. Temporary forks arise when conflicting blocks refer to the same predecessor. Even though temporary forks may occur due to conflicting updates, the blockchain eventually gets synchronized to a single chain, with miners opting for the longest or the heaviest chain [2,27].

In contrast, permissioned blockchains operate in settings where nodes are authenticated and known to the system. These blockchains typically rely on more traditional byzantine fault tolerant consensus protocols such as Practical Byzantine Fault Tolerance (PBFT) [28], which offer fast finality and higher throughput, but assume partial trust among participants. These systems trade decentralization for higher throughput

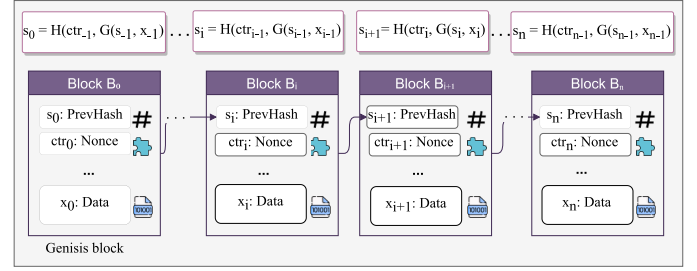


Fig. 1. The main components of blockchain.

and stronger finality, making them suitable for enterprise or consortium deployments.

2.1.1. Immutability pillars

The tamper-resistance of a blockchain rests on three interdependent pillars: cryptographic hash links, Merkle tree, and decentralized consensus. Each mechanism makes unauthorized modifications computationally infeasible and economically unsustainable. Below, we examine how they collectively enforce ledger immutability.

Cryptographic integrity. Blockchain immutability is fundamentally secured via cryptographic primitives that bind the data structure at two levels. At the transaction level, integrity is ensured through *Merkle trees*. Transactions in a block are recursively hashed pairwise in a binary tree structure, producing a single compact hash root that is stored in the block header. Altering a single transaction propagates changes upward through the tree and results in a different Merkle root, thereby invalidating the entire block. At the block level, *cryptographic hash functions* link block headers sequentially. Each block explicitly references the hash of its predecessor, creating a cryptographically enforced chain back to the genesis block. The hash chaining produces a tamper-evident ledger where altering any block's content changes its hash and breaks the cryptographic link for all subsequent blocks. Since hash functions are deterministic, collision-resistant, and highly sensitive to input changes, any unauthorized modification to past content becomes immediately detectable by any node verifying the chain [29].

Consensus Finality. In the absence of a central authority, blockchain systems rely on distributed agreement among nodes to validate transactions and append blocks. The decentralized consensus protocol guarantees that the blockchain remains secure and tamper-proof even in the presence of malicious users or miners. In practical terms, nodes in the blockchain must reach a consensus before adding a new block to the chain. In PoW systems like Bitcoin [2], consensus is achieved through a competitive process where miners expend computational resources to solve cryptographic puzzles. This process ties block creation to substantial computational effort, making fraudulent modifications prohibitively expensive. The high cost of computation and time required for block generation provides strong protection against tampering. Once a block is appended, any adversary attempting to alter the block must re-mine all subsequent blocks faster than the rest of the network, which becomes exponentially harder as the chain grows [30].

State Semantics. Beyond the cryptographic bindings, and consensus protocols blockchain immutability is enforced by state semantics. Fundamentally, the blockchain operates as a deterministic state machine where each transaction constitutes a state transition that moves the system from one valid global state to the next. Transactions are processed in a strict, consensus-validated order to produce a single, canonical global state (e.g., the UTXO set in Bitcoin or account balances in Ethereum) that is verifiable by all participants [31,32]. This ordered sequence forms an append-only execution log. Any honest node executing the same sequence of transactions from the genesis block will derive an identical global state. The integrity of the current state is therefore a direct, verifiable consequence of the entire historical log. Consequently, any attempt

to modify the past violates the sequential logic required to derive the current valid state.

2.2. Motivating factors for redaction

Aside from its original purpose as a digital payment system, blockchain also stores arbitrary data. For instance, in Bitcoin, the `OP_RETURN` opcode allows the storage of arbitrary data as part of a transaction. Matzutt et al. [14] revealed that approximately 1.4% of Bitcoin transactions contained non-financial data, encompassing a diverse range of data types. This poses a considerable threat to the network participants, since they must locally replicate the entire blockchain, including potentially improper content. The authors of [33] list five types of *non-financial* information that can be inserted in the blockchain and pose high risks if possessed by the users of the blockchain network, namely copyright violations, malware, privacy violations, politically sensitive content, and illegal content. Building upon these concrete risks associated with immutable ledgers, we identify the core motivations underlying the need for data flexibility and redaction in blockchain systems.

2.2.1. Regulatory and legal requirements

Modern data protection frameworks such as the European GDPR [34] and China's PIPL [35] mandate that individuals have control over their personal data. However, traditional blockchains structurally violate this principle. Once data is recorded on-chain, it becomes permanently immutable, directly conflicting with regulatory requirements for data removal. In particular, the immutability of data in these distributed ledgers implies that one of the key principles of GDPR, Art.17 Right to erasure, is not met by blockchain, since there is no built-in mechanism to clear personal data upon request.

2.2.2. Ethical considerations

The disclosure or even the mere possession of particular data, including politically sensitive material, pirated data, blasphemy, and hate speech, may be illegal in certain jurisdictions. Due to the decentralized architecture of blockchain systems, nodes may intentionally or inadvertently possess, distribute, or host illegal content embedded within the ledger. The 2018 discovery of child abuse imagery encoded in Bitcoin transactions [33] exemplifies how blockchain neutrality can be exploited. As a consequence, node operators can incur criminal liability or regulatory sanctions solely for storing a canonical copy of the ledger storing illegal content. Beyond legal exposure, the inherent immutability of the records ensures that harmful content remains permanently available, perpetuating reputational and societal damage indefinitely.

2.2.3. Technical motivations

Operational error correction. Complex software systems are inherently susceptible to flaws that can have serious consequences in production environments. This risk is particularly acute in regulated sectors like healthcare and finance, where data accuracy is critical [36]. Erroneous entries, such as incorrect patient records or financial transactions, must be rectifiable. However, blockchain systems amplify this risk because they lack centralized intermediaries that can intervene to correct errors and mitigate damage. Additionally, when a vulnerability is exploited or erroneous data is recorded, the system provides no straightforward mechanism to repair or reverse the damage causing detrimental disruptions to the production chain.

Patching software flaws. Smart contracts flaws pose significant threats to Decentralized Applications (DApps) operating on the blockchain, especially in terms of monetary losses. A single vulnerability can freeze or siphon millions, as demonstrated by the DAO attack [16] or the Poly Network hack [37]. Once deployed, smart contracts cannot be patched, leaving vulnerabilities permanently exploitable [38,39]. Addressing these flaws requires hard forking the network or deploying new contracts. Both approaches are costly and can fragment ecosystems and undermine user trust.

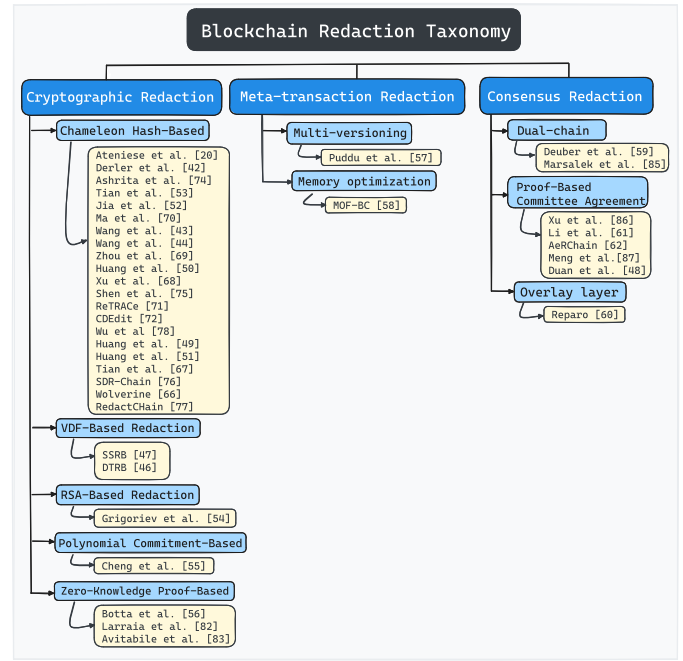


Fig. 2. Taxonomy of state-of-the-art solutions for blockchain redaction.

Storage optimization. The append-only architecture of blockchain creates an unsustainable storage trajectory. The on-chain storage becomes costly and inefficient for maintaining permanent, full copies of often redundant data (e.g., expired contract logs, deprecated tokens, or stale off-chain references) [40]. In 2024, for instance, the Bitcoin ledger size exceeded 600 GB [41], imposing a significant burden on nodes with limited resources and raising operational costs for participants maintaining full archival copies.

3. Taxonomy of redaction techniques

Blockchain redaction refers to the ability to modify blockchain content after it has been committed, without undermining the integrity, auditability, or security guarantees of the ledger [20]. Enabling such functionality requires relaxing one or more of the system properties that collectively enforce immutability. Building on the analysis in Section 2, blockchain immutability rests on three interdependent pillars, namely, *cryptographic integrity*, *consensus finality*, and *deterministic state semantics*. In this section we define a structured taxonomy of redactable blockchain techniques, derived from this perspective. Redaction techniques are classified according to the specific immutability pillar they are designed to modify. The taxonomy comprises three major classes, divided into distinct subclasses based on specific technical mechanisms and control models as illustrated in Fig. 2: *Cryptographic Redaction*, which operates on the cryptographic integrity; *Meta-transaction Redaction*, which modifies the state semantics; and *Consensus-based (Voting) Redaction*, which enforces collective authorization through consensus.

This taxonomy is derived from a comprehensive review of the literature, tracing the evolution of redactable blockchain systems from the seminal work of Ateniese et al. [20] in 2017 to peer-reviewed advancements published through 2025. Our analysis prioritizes works that introduce foundational mechanisms or demonstrate substantial conceptual advances within a paradigm. Incremental optimizations are contextualized within the evolutionary trajectory of their respective classes. The following subsections detail each class, tracing its technical evolution from foundational algorithms to recent enhancements, analyzing operational trade-offs, and discussing hybrid systems that operate at the intersections of these core paradigms.

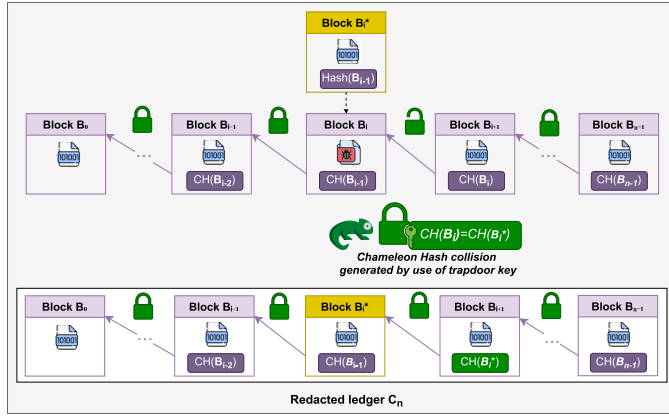


Fig. 3. Chameleon hash blockchain redaction. A block B_i^* is generated to replace the infected block B_i . Leveraging the trapdoor collision property of the Chameleon hash, a collision can be found in block B_i^* such that $CH(B_i) = CH(B_i^*)$. The block B_i^* replaces the block B_i without compromising the hash link between the adjacent blocks.

3.1. Cryptographic redaction

Cryptographic redaction refers to a class of techniques that enable post-hoc modification of blockchain content by embedding mutability into the cryptographic constructs responsible for enforcing block integrity. This class of techniques redefines hash links, commitments, or authentication structures in a way that permits authorized modifications without violating verifiability or breaking chain continuity. Within this class, we identify five prominent cryptographic constructions, including Chameleon Hash-based, VDF-based, RSA-based, Polynomial Commitment, and Zero-Knowledge Proof redaction. Each technique instantiates the redaction capability through different assumptions, offers unique trade-offs in terms of efficiency, security guarantees, and implementation complexity.

3.1.1. Chameleon hash-based redaction

The first technical proposal for a redactable blockchain was introduced by Ateniese et al. [20], which operates on the cryptographic hash binding to demonstrate that cryptographic integrity and controlled mutability could coexist. Their scheme replaces the ledger's standard collision-resistant hash function with *chameleon hash* [42], a primitive that enables controlled mutability while preserving the chain's verifiable structure.

Chameleon hash functions share key properties with standard cryptographic hashes, including preimage resistance and second preimage resistance. These properties ensure that it is infeasible to determine a message that corresponds to a given hash value or discover a second message possessing the same hash as the stated message. The critical distinction lies in the inclusion of a trapdoor key [42]. While chameleon hash functions remain collision-resistant to the public, the possession of the trapdoor key allows the holder to efficiently compute arbitrary collision computation, which essentially breaks the collision resistance of the hash function. In the blockchain context, chameleon hash allows to unlock the blocks hash and replace the block data, while keeping the block hash intact (See Fig. 3).

More concretely, considering the block format defined in (2), the original inner hash function G is replaced by the chameleon hash function $Hash(hk, (s_i, x_i), r)$ with hk being the hash key, and r the randomness for the chameleon hash. This allows the holders of the trapdoor key td to edit the block data x_i without compromising the consistency of the hash link between B_i and B_{i+1} . The new expression of the hash link becomes $s_{i+1} = H(ctr_i, Hash(hk, (s_i, x_i), r))$. This redaction algorithm involves a change in the block structure of the blockchain (defined in expression (1)). A new field is added to the block header to

include chameleon hash randomness r . The block is now in the form $B := (s, x, ctr, r)$. The predicate of validation for a block becomes:

$$Validblock_q^D(B) := (H(ctr, Hash(hk, (s, x), r)) < D) \quad (3)$$

$$\wedge (ctr < q) = 1$$

A set of validators are entrusted with the trapdoor key, which grants them the authority to propose modifications. To redact a block B_i , a validator uses the trapdoor key td to compute a hash collision. The hash collision allows the validator to replace the existing data x_i within the block B_i with the new data x_i^* . The new data x_i^* and the hash randomness r_i^* replace the old data x_i and r_i , respectively, in the modified block B_i^* . Upon completion of the redaction process, the validators broadcast the updated chain as the only valid chain. All network participants are encouraged to adopt the new redacted chain and discard any alternative chains they may have.

Although the original scheme primarily targets a centralized setting with a trusted authority holding the trapdoor, the authors briefly discuss potential extensions to decentralized environments. However, the work does not fully address the coordination and trust distribution challenges required to securely implement redaction in permissionless networks. In particular, in a decentralized setting, the redaction requires mechanisms for key distribution among blockchain miners. The validators reconstitute the trapdoor key using secret sharing schemes [43] and collectively run the redaction algorithm in a distributed manner through a secure multiparty computation (MPC). The basic idea behind the deployed secret sharing scheme [43] is to split the secret td into n shares $td_1, td_2, \dots, td_n$ such that td can be reconstructed only if at least k out of n shares are combined, while the secret remains completely unknown with any $k - 1$ or fewer td_i shares.

While Ateniese et al.'s framework established the feasibility of redaction, the original proposal presented a generalized model that left critical implementation details open to interpretation. A significant vulnerability in this foundational design is the *key exposure problem* [44]. Key exposure refers to the ability of any participant to derive the trapdoor key when the first collision is computed. Such a breach enables unauthorized participants to manipulate the data without leaving any evidence.

To mitigate the risks of permanent trapdoor exposure, Camenisch et al. [45] introduced chameleon hash functions with ephemeral trapdoors (CHET). Crucially, this scheme generates a new, unique ephemeral trapdoor (etd) for each specific chameleon hash invocation (i.e., for every transaction/block), which is combined with a long-term trapdoor held by the authority to compute collisions. When a redaction is required, the transaction owner provides the specific etd associated with that hash to the authority. While the computation of a collision may technically expose the etd for that specific instance, the security model ensures this exposure is contained; since etd values are unique to each hash and cannot be reused, the secrecy of the main trapdoor remains intact, preventing unauthorized modifications to any other part of the ledger.

Many emerging solutions since adopted the CHET construction or introduced novel secure chameleon hash variants. As the field evolved, research progressively shifted from securing the trapdoor to embedding fine-grained control, accountability, and traceability into the redaction workflow. These advancements manifest differently depending on the governance model and trust assumptions of the underlying blockchain. For clarity of exposition, we first examine solutions designed for permissioned environments, then explore those in permissionless settings where decentralized coordination is required.

a) Permissioned and consortium environments.

In permissioned environments, the blockchain is maintained by a defined set of identifiable validators, and redaction control is tightly managed. In this context, the primary challenge is the enforcement of fine-grained access control and accountability among semi-trusted peers.

Derler et al. [46] introduced a Policy-based Chameleon Hash (PCH) which extends the basic chameleon hash primitive with attribute-driven access control. PCH specifically encrypts the ephemeral trapdoor key etd using ciphertext-policy attribute-based encryption (CP-ABE) [47],

embedding a boolean access policy into the ciphertext. CP-ABE provides a more efficient access control mechanism compared to conventional public key cryptographic encryption. Only users whose attributes satisfy the access policy can decrypt *etc.*, and thus compute a valid collision to perform a redaction.

Instead of modifying the hash link of the block, users compute a chameleon hash over the transaction and insert the resulting hash into the Merkle tree. This enables redactions at the transaction level, allowing updates to individual transaction data without affecting the overall block structure or Merkle root integrity. PCH enforces fine-grained access control through attribute-based encryption, ensuring that only users satisfying the embedded policy can decrypt the ephemeral trapdoor and perform redactions. However, it does not guarantee accountability, as redactions are not cryptographically bound to the identity of the modifier once the trapdoor is revealed.

Addressing the accountability gap present in [20] and [46], Ashritha et al. [48] proposed a permissioned block-level solution incorporating mechanisms for accountability and key revocation. Their scheme introduces revocable trapdoors using non-linear secret sharing (NLSS) and distributes redaction authority among validators via MPC. While significantly enhancing security posture, their approach incurred high bandwidth and time costs and still faced key management complexities.

PCHBA by Tian et al. [49] address accountability challenges in PCH schemes, particularly black-box key leakage where modifiers embed keys into external devices for anonymous rewriting. PCHBA introduced black-box accountability enabling an authority to trace redactions back to the responsible modifier even if multiple users share the same access rights. This is achieved using ephemeral trapdoors, homomorphic signatures and a traceable ABE scheme.

The work of Xu et al. [50] also addresses accountability challenges of the PCH frameworks particularly in cases of blackbox key leakage, where modifiers embed their keys into external devices to enable anonymous rewriting. Xu et al. [50] propose a redactable blockchain design built on a Policy-based Chameleon Hash with black-box Accountability (PCHA). In PCHA, transaction owners generate a chameleon hash using CHET and associate it with an attribute-based access policy that is cryptographically bound to a novel Attribute-Based Traitor Tracing (ABTT) mechanism. The ABTT scheme allows a central authority to trace any unauthorized redaction attempt back to all users who contributed to the leaked key, thereby enforcing black-box traceability. In addition, PCHA integrates Identity-Based Signatures (IBS) to bind each redaction operation to a verifiable identity. Modifiers who satisfy the access policy can decrypt the ephemeral trapdoor and compute a valid collision to rewrite the transaction. The use of IBS ensures that every redacted transaction is cryptographically linked to the signer's identity, enabling redaction linkability and accountability.

While PCHBA [63] and PCHA [50] focus on tracing malicious actors after key exposure, Shen et al. [57] takes a complementary approach by preventing unauthorized redactions even when trapdoors are compromised. The core primitive is implemented as a double-trapdoor chameleon hash, where a signing trapdoor used during initial block creation and an editing trapdoor used exclusively for redaction. Even if one trapdoor is leaked, unauthorized redactions remain infeasible, as both are required to forge valid changes. This redaction scheme ensures global state consistency using a cryptographic accumulator that is a compact mathematical structure of valid blocks. When a redaction occurs, it triggers real-time updates to the accumulator and issues publicly verifiable proofs to reflect the changes. Membership proofs confirm active blocks belong to the current chain, while non-membership proofs invalidate redacted or historical versions.

Besides detecting unauthorized redactions, ReTRACe [58] enables dynamic revocation of modification rights. ReTRACe composes three novel cryptographic primitives, including a revocable chameleon hash that extends CHET to ensure that a revoked user cannot modify a blockchain message or trapdoor. Additionally, a revocable CP-ABE scheme encrypts ephemeral trapdoors under access policies. Authorized

users can anonymously redact transactions while their identities remain conditionally traceable and can be revealed by authorized oversight entities when accountability is required.

Another aspect of redaction privilege management was proposed in CDEdit [59] establishing usage constraints of redaction capabilities. CDEdit [59] proposes a token-governed redactable blockchain scheme that emphasizes controllable privilege delegation and multi-level editing. Specifically, it introduces a Privileged Token Service (PTS) to issue edit tokens limiting both the granularity (Tx-level vs. Block-level) and number of permitted edits (n-times per token) per modifier. It combines CHET with a fast and low overhead attribute-based encryption scheme [67] to encrypt the ephemeral trapdoor under fine-grained policies.

Ma et al. [52] propose a decentralized redactable blockchain (DPCH) to overcome the trust and centralization issues inherent in prior PCH systems. DPCH decentralizes redaction control by distributing both the chameleon hash trapdoor and access policies across multiple independent authorities through a multi-authority policy-based chameleon hash (MA-PCH) construction. The scheme combines three cryptographic primitives: a chameleon hash for mutable transaction rewriting, digital signatures for authentication and integrity, and multi-authority attribute-based encryption (MA-ABE) for fine-grained, decentralized access control. Under MA-PCH, no single authority can unilaterally authorize or execute redactions, at the cost of non-trivial coordination and communication overhead during redaction operations.

A recent work of Wang et al. [53] proposes TPRB, a permissioned redactable blockchain that mitigates the key-exposure risk in policy-based Chameleon Hash schemes by distributing redaction authority across multiple entities. The system relies on a novel RSA-based threshold policy-based Chameleon Hash (TPCH) that eliminates reliance on a single trapdoor holder. In TPCH, the long-term trapdoor is decentralized among n authorities using Shoup's integer-based secret sharing [68], such that at least $t + 1$ shares are required to enable collision generation. In parallel, the ephemeral trapdoor is protected through ciphertext-policy attribute-based encryption (CP-ABE). A modifier must both decrypt the ephemeral trapdoor under the CP-ABE policy and obtain the required threshold of shares to perform a rewrite. Then the shares are combined with Lagrange interpolation to produce a valid TCH randomness for the collision, ensuring that no single entity ever controls the full trapdoor and significantly reducing key-exposure risks.

Another work of Wang et al. [54] identifies a critical limitation in policy-based Chameleon Hash schemes, where a malicious user may encrypt an invalid trapdoor under attribute-based encryption, preventing authorized modifiers from performing legitimate edits. To address this issue, they propose a Resilient Redactable Blockchain (RRB). RRB adopts a two-level rewriting model. The first level enables standard transaction-level redactions using PCH. If that fails due to an invalid trapdoor, the system falls back to a centrally controlled block-level rewrite to force the correction. Moreover, RRB incorporates explicit version detection, embedding version identifiers that allow nodes to detect outdated states, thereby preventing reversion attacks and long-term ledger inconsistencies.

Jia et al. [51] propose a novel chameleon hash redaction scheme that supports both regulatory oversight and user self-redaction in a unified redactable blockchain framework. Their construction introduces the stateful Chameleon Hash with Revocable Subkey (sCHRS), a cryptographic primitive that distinguishes between master and subordinate trapdoor keys. The regulator holds the master key, while users generate subordinate keys that can be revoked if abused. To enhance control and reduce unintended alterations, the scheme restricts redactions to specific transaction fields such as OP_RETURN or coinbase outputs where arbitrary data is typically inserted, limiting flexibility by preventing edits to other data segments even when those might require correction. This field-level restriction simplifies policy enforcement but constrains the scope of permissible redactions and may leave critical errors unaddressable.

Table 2
Chameleon hash-based redaction methods (Permissioned).

SOLUTIONS	MECHANISMS	NETWORK	GRANULARITY	MAIN FEATURES
ATENIESE et al. [20]	CH, LSS, MPC and PKE	Permissioned & Permissionless	Block	+ Negligible redaction time – Key exposure – Transactions inconsistency – No Accountability or traceability – Prone to double-spending
DERLER et al. [46]	CHET and CP-ABE	Permissioned	Tx	+ Anti double-spending + User-managed redaction + Accountability – Cost and time overhead – No public traceability
ASHRITHA et al. [48]	CHET, NLSS, DS and MPC	Permissioned	Block	+ Accountability + Key revocation – High bandwidth and time cost
TIAN et al. [49]	CHET, CP-ABE and DS	Permissioned	Tx	+ black-box accountability + Traceability
JIA et al. [51]	CHET	Permissioned	Tx-field	+ User-managed redaction. + Decentralized redaction. + Key revocation. + GDPR-aligned. – Scalability issues.
MA et al. [52]	MA-PCH	Permissioned	Tx	+ User-managed redaction. + Decentralized redaction.
TPRB [53]	TPCH and CP-ABE	Permissioned	Tx	+ User-managed redaction. + Key exposure free. + Decentralized redaction. – Scalability issues.
RRB [54]	TPCH	Permissioned	Tx & Block	+ Key exposure free. + Decentralized redaction. + GDPR aligned. + Post-redaction consistency.
ZHOU et al. [55]	ID-based CH Homomorphic secret sharing	Permissioned	Tx & Block	+ Accountability & Traceability. + User-managed redaction. + GDPR-aligned + Dependency-aware redaction. – Scalability issues.
HUANG et al. [56]	CH and ASCS	Permissioned	Block	+ Accountability. + Key-exposure free. – Redaction privileges misuse. – High bandwidth cost. – Reduced redaction efficiency.
XU et al. [50]	CHET and ABTT and IBS	Permissioned	Tx	+ Accountability and black-box traceability – Trusted central authority – Cost and time overhead
SHEN et al. [57]	Double-key CH, NIZK	Permissioned	Block	+ Full editability + State verifiability + Incentivized redaction adoption – Computational overhead
RETRACE [58]	RCHET and RFAME	Permissioned	Tx	+ Revocation Support + Accountable anonymity – Complex infrastructure
CDEDIT [59]	double-key CH	Permissioned	Tx & Block	+ Incentive-driven + Resistant to collusion
Wu et al. [60]	Lattice-based CH	single-trapdoor Permissioned	Tx	+ Post-quantum secure + Key-exposure-free – Cost and time overhead

Abbreviations. **CH**: Chameleon Hash. **CHET**: Chameleon Hash with Ephemeral Trapdoors. **CP-ABE**: Ciphertext-Policy Attribute-Based Encryption. **DCP-ABE**: Decentralized CP-ABE. **MA-PCH**: Multi-Authority Policy-based CH. **TPCH**: Threshold-PCH. **ABET**: Attribute-Based Encryption with black-box Traceability. **MPC**: Multi-party Computation. **LSS**: Linear Secret Sharing. **NLSS**: Non-Linear Secret Sharing. **Homomorphic SS**: Homomorphic Secret Sharing. **ASCS**: Accountable & Sanitizable Chameleon Signatures. **IBS**: Identity-Based Signature. **NIZK**: Non-interactive zero-knowledge proofs. **DS**: Digital Signature.

Table 3
Chameleon hash-based redaction methods (Permissionless).

SOLUTIONS	MECHANISMS	NETWORK	GRANULARITY	Main Features
HUANG et al. [61]	CHET	Permissionless	Block	+ Hash revocation. – Scalability issues.
HUANG et al. [62]	TUCH and LRRS	Permissionless	Block	+ Detectable double-spending. + Accountability and traceability. + Anonymity. – Computational overhead.
TIAN et al. [63]	CHET, ABET and DPSS	Permissionless	Tx	+ black-box accountability. + Traceability
SDR-CHAIN [64]	CHET, DCP-ABE, LSS	Permissionless	Tx	+ Decentralized redaction + Backward/forward security + Dynamic participation
WOLVERINE. [65]	NITCH	Permissionless	Tx-field	+ Transaction consistency + Accountability + Supports < 1/2 adversaries – Storage Overhead
REDACTCHAIN [66]	CH, DKG	Permissionless	Tx	+ Incentive-driven + Adversarial resilience

Abbreviations. **TUCH**: Time-Updatable Chameleon Hash. **NITCH**: Non-Interactive Threshold Chameleon Hash. **DCP-ABE**: Decentralized CP-ABE. **ABET**: Attribute-Based Encryption with black-box Traceability. **LSS**: Linear Secret Sharing. **NLSS**: Non-Linear Secret Sharing. **DPSS**: Distributed Proactive Secret Sharing. **LRRS**: Linkable & Redactable Ring Signature. **DKG**: Distributed Key Generation.

Building on the same GDPR-driven goal of combining user-initiated redactions with regulator oversight, Zhou et al. [55] propose a redactable blockchain built on an identity-based trapdoor hash without embedding explicit access policies. Instead, redaction rights are enforced implicitly through identity roles. The system supports user-initiated edits for their own transactions, and regulator-initiated deletions or dependency updates. Each user's ability to edit is cryptographically bound to a private key derived from their identity by the regulator. This ensures users can only modify their own transactions. Regulators hold the master secret to supervise the data on the chain. Regulator redaction privileges are collectively managed through a homomorphic secret-sharing scheme, which distributes the master key among multiple authorities ensuring mutual supervision between regulators. All redactions require network consensus and are publicly verifiable via on-chain metadata and consensus logs. While the approach by Zhou et al. offers one of the most controlled and regulation-aligned redaction mechanisms, this strict enforcement comes at the cost of increased system complexity, as each redaction requires an additional layer of consensus and coordinated approval, potentially impacting efficiency in practice.

While prior solutions ignored quantum threats, Wu et al. [60] present two chameleon hash schemes one without lattice trapdoors and another with trapdoors. Both constructions rely on lattice-based assumptions marking a quantum-secure design. Wu et al. address two critical gaps in redactable blockchains, first, the problem of key exposure, and second, the quantum vulnerability of existing chameleon hashes that rely on discrete-log/factoring assumptions. In the proposed design, each transaction or block is associated with a chameleon hash value, generated using a lattice-based one-way function. When a redaction is needed, any party possessing the public trapdoor can compute a collision.

The first construction assumes that the trapdoor is made fully public. In this scheme, anyone can compute collisions, but redactions are controlled externally through a voting mechanism. Wu et al. [60] also propose a distributed variant where the trapdoor is shared among multiple parties using secret sharing or lattice-based threshold trapdoor generation. In this version, redactions are only possible if a threshold number of parties cooperate to reconstruct a valid short preimage, enabling committee-based redaction.

b) Permissionless environments

In permissionless blockchains, redaction authority is distributed among

anonymous, dynamically changing participants without relying on trusted third parties or fixed governance structures.

Huang et al. have produced a sequence of redactable blockchain schemes for resource-constrained industrial IoT environments, starting with [56] that supports group-based redaction in consortium blockchains while maintaining accountability and per-block signature sanitization for IoT contexts. Huang et al. further advanced this concept toward permissionless settings and proposed a self-redactable blockchain architecture that enables localized redactions through collaborative decision-making among sensors [61]. Building on this direction, their later work introduced the Time Updatable Chameleon Hash (TUCH) [62], which constrains collision validity to a predefined time window. This temporal restriction ensures that even if a trapdoor key is compromised, it becomes unusable after expiration, reducing attack windows and mitigating long-term key exposure. In this design, the miner generates the trapdoor key and thus temporarily holds redaction authority. Importantly, TUCH avoids trapdoor key sharing and coordination overhead by allowing spontaneous ring formation for redactions and integrates seamlessly with Linkable and Redactable Ring Signatures (LRRS). LRRS enables miners to modify transaction content without invalidating signatures, while preserving signer anonymity and linkability to detect double-spending, making the approach suitable for dynamic, trustless IoT environments.

Tian et al. also extended their previous policy-based proposal to a fully decentralized and permissionless setting [63] by eliminating reliance on a trusted authority through dynamic proactive secret sharing (DPSS). The decentralized scheme distributes trapdoor key shares across dynamically rotating committees, requiring a threshold of committee members to authorize each redaction. Transactions embed fine-grained access policies enforced through attribute-based encryption, allowing only authorized modifiers to compute hash collisions to rewrite content while preserving chain integrity. Crucially, the scheme incorporates a publicly traceable attribute-based encryption (ABET) mechanism that enables anyone to verify and trace modified transactions back to the specific modifier and authorizing committee. Every redaction is signed and publicly accountable, ensuring that modifications are both controlled and verifiable in an open blockchain environment.

Extending the committee-based model, SDR-chain [64] introduces a more flexible approach by enabling dynamic participation of both

redactors and attribute authorities. SDR-chain leverages a customized chameleon hash construction that supports dynamic participation, allowing attribute authorities to join and leave without compromising functionality or security. To manage redaction keys securely SDR-chain employs decentralized CP-ABE, allowing attribute-based trapdoor generation and delegation across a dynamic set of authorities. Notably, SDR-chain ensures both backward and forward security, which guarantee that the compromise of an authority does not expose past redactions nor impact future ones.

To eliminate interactive coordination entirely, Wolverine [65] introduces Non-Interactive Threshold Chameleon Hash (NITCH) scheme, where each committee member independently computes a partial collision share using a private trapdoor fragment. NITCH enables collision generation without interaction among committee members. Each member locally computes a collision share based on their private trapdoor fragment. Any participant can later aggregate the shares to reconstruct the full collision. A regulatory committee elected via Sybil-resistant mechanisms collectively holds shares of the NITCH trapdoor key. Each committee member holds a secret share (sk_i) of the master trapdoor, and collisions require $t + 1$ shares ($t < n/2$), preventing minority control. Any user can initiate a redaction request by embedding it into a transaction. The committee evaluates the request according to redaction policy, ensuring that modifications target only admissible fields and preserve transaction consistency. Upon approval, each committee member independently computes a partial collision share and broadcasts it as an on-chain vote transaction. Miners aggregate valid votes into blocks. Once a majority of committee votes approve the request, shares are combined to create a valid hash collision and execute the redaction.

Wolverine enforces transaction consistency throughout this process. It models dependencies between transactions formally and prevents any redaction that would violate semantic validity or break links in subsequent blocks. If a redaction affects transactions with dependencies, it is either rejected or requires coordinated cascading redactions to preserve integrity. Finally, to ensure long-term security and mitigate trapdoor compromise, Wolverine periodically rotates the redaction committee using randomness from Decentralized Random Beacons (DRBs) [69] derived from NITCH ensuring that redaction authority is periodically reassigned.

RedactChain [66] presents a hybrid approach combining threshold chameleon hash functions (TCHF) with a jury-based voting mechanism. Redaction authority is decentralized through a periodically rotated jury, whose members collaboratively generate chameleon hash collisions using a distributed key generation (DKG) protocol. Redactions are permitted only within a defined window during the jury's tenure, ensuring time-bounded privileges and limiting the risk of misuse. The system maintains an immutable redaction log for transparency and distinguishes between removal for non-spendable outputs and obfuscation for spendable outputs to preserve transaction validity. Furthermore, The design provides formal adversarial resilience, tolerating attackers with up to 25% of mining power. Despite its robustness, the approach remains tightly coupled to the Bitcoin transaction model and introduces coordination overhead due to jury management and voting synchronization.

3.2. VDF-based redaction

Most cryptographic redactable blockchain schemes rely on chameleon hashes. While these approaches allow modifications without breaking hash links, they generally fail to enforce global consistency across all nodes, as they rely on voluntary adoption. Wang et al. first proposed SSRB [70], a redactable blockchain for permissioned settings that addresses these consistency issues using Verifiable delay functions (VDFs). VDF-based redaction introduces a different paradigm by treating time as a verifiable resource. A VDF requires sequential computation over a specified time parameter, and produces a proof that can be efficiently verified by any node. Under normal

operation, blocks are generated with a fixed time difficulty, ensuring sequential computation. Redaction in SSRB is realized through chain reconstruction. When a redaction is authorized, a trusted authority holding the VDF trapdoor can quickly recompute the VDF outputs for the modified block. All subsequent unmodified blocks must be recomputed to maintain hash linking, forming a reconstructed chain. The reconstructed chain has the same length as the original chain but a larger cumulative VDF time value, since redacted blocks are assigned a higher effective time parameter. SSRB enforces synchronized redaction through a longest-time rule; nodes must adopt the reconstructed chain to remain on the main chain. Nodes that refuse to synchronize cannot validate or extend future blocks, since blocks extending stale history reference a hash pointer different from that of the reconstructed chain head. However, SSRB relies on a centrally held VDF trapdoor, introducing a single point of trust.

Building on SSRB, the authors later introduced DTRB [71], which replaces the single trapdoor with a threshold-shared trapdoor VDF (DTVDF). In DTRB, the secret VDF trapdoor is split into shares distributed among multiple participants. Redaction requires collaboration from a threshold of nodes to reconstruct the trapdoor and accelerate the VDF computation thus it ensures that no single entity can unilaterally modify the chain. Redaction in DTRB proceeds as follows. After a redaction proposal is approved, the leader node identifies the earliest block to be modified and initiates collaborative chain reconstruction. For each affected block, participating nodes independently compute DTVDF result shares along with proofs of correctness. The leader then combines a sufficient set of shares via Lagrange interpolation to reconstruct the accelerated VDF output and rebuild the updated chain segment. Once the modified chain segment is reconstructed, DTRB further improves efficiency through aggregate verification. Instead of verifying each redacted block's VDF proof individually, DTRB introduces an aggregated proof for the entire reconstructed chain at once and provides accountability by embedding the modifier's signature into aggregated proofs. Overall, DTRB maintains the consistency guarantees of SSRB while restoring decentralization.

3.2.1. RSA-based redaction

RSA-based constructions provide another cryptographic route to redactable blockchains, replacing hash-linking with RSA bindings to balance integrity and controlled redaction. Grigoriev et al. [72] propose an RSA-based redactable blockchain scheme based on the computational hardness of the RSA problem. The approach replaces the standard hash-link with RSA-based bindings, where unauthorized tampering would require solving infeasible problems such as RSA inversion or discrete logarithms.

Each block contains three core components: a permanent prefix (ensuring immutability of structural links), modifiable content, and a redactable suffix (enabling authorized edits). The permanent prefix P_i is a fixed value derived from the previous block's hash, ensuring unbreakable links between blocks $P_i = H(P_{i-1}, C_{i-1})$ with H is a collision-resistant hash function. The modifiable content C_i represents the editable data. The redactable suffix X_i is an RSA-based value that secures and verifies authorized edits. Blocks are linked through a hash function applied to the prefix and content, generating a hash value. The suffix is calculated by converting h_i into an integer d_i modulo n (where $n = pq$ is the product of two large primes p and q).

A central authority possesses the RSA private key, which consists of two critical secret components: (i) Prime factors p and q of the modulus $n = pq$. (ii) The function $\phi(n) = (p - 1)(q - 1)$. These elements enable the authority to compute modular inverses and generate valid suffixes during redactions. Technically, the RSA private key is used to derive e_i a modular inverse of d_i such as $e_i \equiv (d_i)^{-1} \pmod{\phi(n)}$. The suffix X_i is then calculated such as $X_i \equiv P_{i+1}^{e_i} \pmod{n}$ where P_{i+1} is the prefix of the next block. This enforces that:

$$X_i^{d_i} \equiv P_{i+1} \pmod{n} \quad (4)$$

This congruence ensures that only valid suffixes (computed with d) can satisfy the relationship to P_{i+1} .

To redact a block, a trusted authority replaces the original content C_i of a block B_i with new content C_i^* , recomputes the hash $h_i^* = H(P_i, C_i^*)$, derives a new integer d_i^* , find e_i^* , the modular inverse of d_i^* and adjusts the suffix $X_i^* \equiv P_{i+1}^{e_i^*} \bmod n$ using the RSA private key. Critically, the prefix P_{i+1} remains unchanged, preserving the link to subsequent blocks.

$$(X_i^*)^{d_i^*} \equiv P_{i+1} \bmod n \quad (5)$$

The security of RSA-derived method stems from the fact that recovering X_i^* from P_{i+1} and d_i^* without knowing p , q , or d is computationally infeasible (equivalent to solving the RSA problem). Also, Since P_i depends on prior blocks, tampering propagates detectably. Hence, if an adversary tries to revert to C_i and X_i , they must modify the block's current state, which would break the link: $(X_i)^{d_i^*} \not\equiv P_{i+1} \bmod n$ since $d_i^* \neq d_i$. However, if a malicious redactor maintains a private log of prior block content C_i and suffix X_i , they can revert the blockchain to any logged state by reapplying the redaction process [73].

3.2.2. Polynomial commitment-based redaction

The polynomial-based blockchain introduces a novel structure where each block is cryptographically linked through polynomials rather than hashes [74].

For each field in the block (header or data), a hash-cut function HASHCUT is applied to produce a unique 2D coordinate (x_i, y_i) . This function first hashes the data value with other metadata and padding, then extracts numeric values from the hash output $x_i = \text{HASHCUT}(H(tx_i \parallel \text{metadata} \parallel \text{padding}))$ and y_i is chosen from a specific modification difficulty range. For the header specifically, the coordinate (x_0, y_0) is derived by applying HASHCUT to the hash of the previous block's polynomial $L_{i-1}(x)$, metadata and padding $(x_0, y_0) = \text{HASHCUT}(H(L_{i-1}(x) \parallel \text{metadata} \parallel \text{padding}))$. By mapping each transaction to its corresponding coordinate, we get a set of labeled points $\{(x_0, y_0), (x_1, y_1), \dots, (x_k, y_k)\}$. Using these points, a unique interpolation polynomial $L_i(x)$ is constructed using Lagrange interpolation. The resulting polynomial $L_i(x)$ is a cryptographic fingerprint of block B_i , replacing the block hash. The next block B_{i+1} includes in its header a point (x, y) such that $L_i(x) = y$. This process is similar to referencing the hash of the previous block in traditional blockchains. Thus, blocks are linked via polynomial consistency; each new block includes a point from the polynomial defined by the previous block. If the polynomial is tampered with, the link breaks.

Modifications require recalculating these coordinates to satisfy the polynomial relationship, ensuring continuity in the chain. The system allows users to define a modification difficulty that refers to a range or set of valid ordinate values ($y \in [a, b]$) that new coordinates must satisfy during redaction.

Redaction in this scheme is achieved by carefully modifying a field (data or header) to a new value tx^* . The system then must re-map this new value to a new coordinate (x^*, y^*) via a hash-cut operation, such that x^* is unique and does not collide with existing coordinates. Most importantly, the coordinate (x^*, y^*) must still lie on the existing polynomial curve of $L(x)$ such that $L(x^*) = y^*$. This ensures the redacted block is still consistent with the polynomial structure of the chain. To avoid trivial tampering, the system implements a "modification difficulty" constraint that restricts valid y^* values to a specific range $[a, b]$. This mechanism acts like a PoW puzzle, requiring computational effort to find valid replacement coordinates that maintain polynomial consistency across the blockchain while satisfying the difficulty constraint.

The ability to set modification difficulty allows the same blockchain structure to serve applications with varied security requirements. Furthermore, the difficulty parameters allow users to enforce a high degree of control over their data. Transactions can either be set immutable by configuring difficulty parameters to require infeasible computation or support potential redaction with lower settings. Nonetheless, such

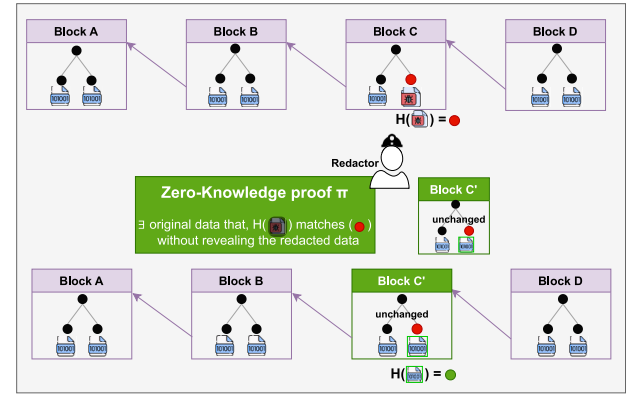


Fig. 4. Zero-knowledge proof based redaction. After redacting block C, the old hash is kept on the merkle tree. The redactor generates a zero-knowledge proof π that proves the existence of original data such that the hash of the redacted data matches the original hash, without revealing what was redacted.

a mechanism requires knowledgeable users of the impact of this difficulty parameters to choose the value that properly balances security with redaction rights.

3.2.3. Zero-knowledge proof-based redaction

In Botta et al. [75], the authors propose a framework that enables Bitcoin nodes to redact specific non-financial data from transactions using zero-knowledge proofs. The technique targets auxiliary data, such as information embedded in coinbase or OP_RETURN outputs, without affecting the financial semantics or exposing the system to double-spending vulnerabilities. Technically, the mechanism employs zero-knowledge proofs to prove partial knowledge of hash preimages corresponding to the redacted data, without revealing the deleted data.

A redactor N identifies a transaction tx containing data to delete. The redactor creates a modified transaction tx^* by replacing sensitive data with zeros. Also, it retains the original hash $h = \text{Hash}(t)$, which serves as a cryptographic commitment to the unaltered data.

The redactor N replaces tx with tx^* in its local blockchain copy. Fundamentally, the Merkle tree root in the block header remains unchanged. Instead of recomputing the root after modifying tx , N keeps the original hash h in the Merkle leaf for tx^* . Additionally, N generates a non-interactive zero-knowledge proof (NIZK) π corresponding to this redaction for verification. The proof π proves that there exists some original data that, when inserted into tx^* , produces a transaction tx^{**} whose hash matches the original hash h . In other words proof ensures that tx^* is a legitimate redaction of tx , even though the original data is hidden. N replaces tx with tx^* in its local copy of block B and deletes the original redacted data. Fig. 4 illustrates the process of redaction.

When another node P requests block B , the redactor N sends the modified transactions tx^* with the original hashes h and the NIZK proofs π . When P receives a redacted block B , it initiates a verification procedure to ensure the validity and integrity of the redacted data. First, P verifies the zero-knowledge proof π associated with the redacted transaction tx^* . This verification confirms that the original hash h corresponds to a valid transaction and that the redacted bytes are confined to permissible positions. Next, P reconstructs the Merkle tree in such a way, for unmodified transactions, it uses their actual transaction hashes as Merkle leaves and for redacted transactions, it substitutes the original hash h (extracted from the NIZK proof statement) as the corresponding leaf. Finally, P computes the Merkle root from these leaves and checks its consistency with the root included in the header of block B . If the computed root matches the header root, the redacted block B is deemed valid.

The proposed data redaction solution offers a balance between blockchain immutability and regulatory compliance by enabling nodes

to delete specific data while preserving the consistency of the chain. A distinguishing strength of this approach is its post-quantum security, which ensures that redacted content remains protected against future cryptographic vulnerabilities. Additionally, the solution supports local redaction autonomy, allowing individual nodes to act without consensus, and cryptographic transparency via zero-knowledge proofs that validate deletions without revealing sensitive data. The solution maintains public verifiability, ensuring that all transactions remain consistent and independently verifiable after redaction, which preserves the foundational trust mechanism of distributed ledgers.

While the solution effectively addresses regulatory tensions and ethical considerations by providing a mechanism to remove illicit content, it fails to align with the storage constraints inherent to blockchain systems. Specifically, generating and storing cryptographic proofs introduces computational and storage overhead. Additionally, the lack of global coordination in the redaction process risks creating fragmented versions of the blockchain across different nodes.

Another improvement by Larraia et al. [76] offering formal security guarantees and technical improvements over Botta's solution [75]. The blockchain is partitioned into equivalence classes based on system-wide policies. Two blockchains belong to the same class if they agree on critical validation data such as the UTXO state, but they may differ in non-essential, redactable data. Nodes can store different versions of the blockchain as long as they belong to the same class.

Building on Ateniese's redaction framework [20], the authors of [77] extend the concept to address state consistency issues introduced in smart-contract-enabled, permissioned blockchains. Their work specifically targets the challenge of updating transactions without compromising the correctness of smart contract execution or the integrity of the global state. The proposed approach leverages the zero-knowledge-based redaction techniques from [75] to construct proofs-of-consistency that validate post-redaction state correctness. The redaction process begins with the identification of the transaction to be modified. Using chameleon hash functions, the redacting authority constructs a substitute block where the targeted transaction tx is altered or removed. The system then identifies all smart contract states and subsequent transactions that depend on tx . For each affected transaction, the redacting entity generates SNARK proofs attesting that state transitions remain valid despite the redaction. These proofs certify that (i) the original transaction tx produced a valid intermediate state st' , (ii) subsequent transactions $\tilde{tx}$ validly transitioned the state from a state st' to a final state st'' , and (iii) the chameleon hash of the modified block remains consistent with the original block hash recorded on-chain.

This framework enforces modifications to directly affected transactions, while all dependent transactions are subsequently validated through their attached proofs. However, this scheme is most effective for redactions that involve deletions or neutral updates; any change to transaction semantics risks violating smart contract logic and causing inconsistencies. Additionally, the method presumes a trusted governance model to manage trapdoors and proof generation, which limits its applicability in permissionless blockchain environments.

Summary. Cryptographic approaches achieve redaction by fundamentally augmenting or replacing the immutable hash linkage of conventional blockchains with specialized primitives. Most prominently chameleon hashes, but also RSA-based commitments, polynomial commitments, and zero-knowledge proof schemes. These techniques embed a controlled trapdoor key (often distributed via secret-sharing or attribute-based encryption) that permits authorized collision-finding and block or transaction rewrites without breaking the inter-block hash chain. Variants address key-exposure (ephemeral or time-limited trapdoors), accountability (traceable ABE, linkable signatures), and quantum resistance (lattice-based constructions). While offering fine-grained redaction and strong cryptographic guarantees, they incur nontrivial overhead in key management, proof generation, and validation complexity, and often require permissioned or semi-permissioned gover-

nance models. To facilitate comparison across reviewed proposals, we summarize their main features in Tables 2 and 4.

Cryptographic redaction approaches enable direct modification of blockchain content by altering its underlying immutability primitives. These solutions provide strong cryptographic guarantees. They preserve the verifiability of redacted chains, ensure auditability, and allow fine-grained redaction control through trapdoor mechanisms or policy-driven encryption. In particular, they support transaction or field level mutability while maintaining the structural integrity of the chain, making them well-suited for sensitive applications in regulated or high-assurance environments. However, these benefits come at a cost. The implementation of such mechanisms often introduces substantial complexity in cryptographic design, key management, and coordination protocols, especially in decentralized settings. Some schemes risk centralization if redaction authority is poorly distributed, while others suffer from scalability limitations due to computational or storage overhead. These challenges have motivated the exploration of alternative paradigms that prioritize simplicity, operational efficiency, and policy-driven control. Notably, meta-transaction-based approaches shift the redaction logic from cryptographic mutation to logical reinterpretation, achieving redaction through transaction versioning and access control without modifying the underlying block structure.

3.3. Meta-transaction based redaction

In contrast to cryptographic solutions that modify block contents through specialized cryptographic schemes, meta-transaction-based redaction operates through semantic override. This approach preserves the strict data immutability of the ledger while enabling logical modifications through controlled state interpretation. Redaction is achieved by appending explicit corrective transactions that reinterpret specific past entries without physically altering them. Consequently, the modification occurs strictly at the state-interpretation level; the original data remains intact, but the canonical view of the state is updated to reflect the redaction. In the following, we examine representative techniques in this class and analyze their mechanisms, guarantees, and limitations.

A foundational system in this class is μ chain proposed by Puddu et al. [78]. μ chain stores multiple versions of data, but designates only one as the active, valid state. Redactions are executed through special mutation transactions that alter which version is considered active. The outdated version persists on the chain but is cryptographically obscured, as validators stop distributing the keys needed to access it.

Formally, in μ chain, each transaction is defined as a triplet (T, a, P) , where T is a transaction set containing multiple encrypted versions τ_i , a is the index of the currently active (decrypted) version, and P is a mutation policy specifying who can issue redactions or extensions, and under what conditions. A mutant transaction (Ref_T, a') activates a different transaction version by referencing an existing transaction set and updating the active index. To add new alternatives, an extension transaction (Ref_T, T_x) appends new versions $T_x = \tau_{k+1}, \dots, \tau_m$ to the original set. Special *nope transactions* also allow marking a transaction as logically deleted.

When a mutable transaction T is sent, the sender specifies the active transaction index a in the transaction set τ and provides a mutation policy P for the mutable transaction indicating how it may be modified (Fig. 5 step 1). Upon receiving transactions, miners add them to the blockchain and make only the active transaction available by decrypting it (Fig. 5 step 2). Subsequently, a mutator M , determined by the mutation policy of the submitted transaction T , submits a mutant transaction T' . The mutant transaction T' contains a reference to the transaction to be modified Ref_T and indicates a new active transaction version $a' \neq a$ from the transaction set (Fig. 5 step 3). Once the new version is validated by the set of validators, it is finally added to the next block, making the decryption keys of previous versions unavailable (Fig. 5 step 4). Similarly, authorized users can extend the transmitted transactions (w.r.t., the mutation policy) by issuing an extending transaction.

Table 4
Cryptographic-based redaction methods (continued).

SOLUTIONS	MECHANISMS	NETWORK	GRANULARITY	MAIN FEATURES
SSRB [70]	VDF	Permissioned	Block	+ Strong synchronization. + Support dynamic nodes. – Centralized redaction. – Computational overhead.
DTRB [71]	DTVDF & NIZK	Permissioned	Block	+ Strong synchronization. + Accountability + Decentralized redaction. – Secret-sharing overhead.
GRIGORIEV et al. [72]	RSA	Permissioned	Block	+ Strong cryptographic security. + Anti double-spending. – Centralized redaction authority. – No accountability or public traceability. – Key Management problems.
CHENG et al. [74]	Lagrange polynomials	Permissioned	Block	+ Adjustable Security. + User-managed redaction. – Not compatible with Competitive consensus (e.g., PoW). – No accountability or public traceability. – Storage Overhead. – Computational overhead for high-degree polynomials.
midrule BOTTA et al. [75]	zk-SNARK	Permissionless	Tx-field	+ Adjustable Security. + Post-quantum secure. + Data Consistency. + Redaction accountability and traceability. – Bitcoin-specific. – Limited fields for redaction. – Storage Overhead.
LARRAIA et al. [76]	zk-SNARK	Permissionless	Tx-field	+ Enhanced Security. + Post-quantum secure. + Data Consistency. + Redaction accountability and traceability. + Generalized to PoW. – Limited fields for redaction. – Storage Overhead.
AVITABILE et al. [77]	CH & zk-SNARK	Permissionless	Block	+ Chain Consistency. + Redaction accountability and traceability. + Public Verifiability. – Centralized Governance. – Key Management. – Storage Overhead.

Abbreviations. **VDF**: Verifiable Delay Function. **DTVDF**: Decentralized trapdoor VDF. **RSA**: Rivest-Shamir-Adleman. **zk-SNARK**: Zero-knowledge succinct non-interactive arguments of knowledge.

The μ chain is presented in both permissioned networks, where designated validators manage decryption keys for active transactions, and permissionless networks, which require a secret sharing scheme for key distribution. To handle dynamic validator groups and network churn, μ chain integrates a dynamic proactive secret sharing (DPSS) scheme [79] alongside Shamir's secret sharing scheme [43].

The μ chain offers a high level of transaction consistency post-redaction. The system processes the history of a mutable transaction set and their dependencies; when a transaction is modified, the system automatically triggers the necessary updates of dependent transactions to maintain validity and prevent double-spending, ensuring transaction consistency.

More pronouncedly, μ chain scheme does not edit the block data containing the old transactions, therefore no modifications to the block hash are required. Modified transactions are generated and subsequently recorded in next blocks. As a result, obsolete transactions remain on-chain in encrypted form, hidden but not removed. While this approach maintains blockchain integrity, it incurs storage overhead due to persisting encrypted inactive transactions indefinitely. Furthermore, reliance

on multiple encryption keys necessitates complex secure key management (distribution, rotation, revocation), impacting scalability and performance through added cryptographic operations.

Another work of Dorri et al. [80] presents MOF-BC a blockchain redaction framework designed to address storage scalability and privacy challenges in IoT networks. MOF-BC [80] permits users to delete or summarize their own transactions, enabling a form of “*right to be forgotten*” while preserving overall consistency.

To enable redaction without compromising privacy or requiring long-term key storage, MOF-BC introduces the Generator Verifier (GV). The GV is a unique identifier for each transaction, created by signing the hash of the previous transaction identifier $P.T \times ID$ which establishes the link between successive transactions created by the same node. And a Generator Verifier Secret (GVS) corresponding to some user-held secret. All GVs are linked to a single public key ($GV-PK^+$), so users only need to maintain one private key. In practice, when a user wants to remove a transaction they created, they issue a special *redacting transaction*. The *redacting transaction* records the hash of the deleted transaction and the hash of the previous transaction to ensure the transaction consistency.

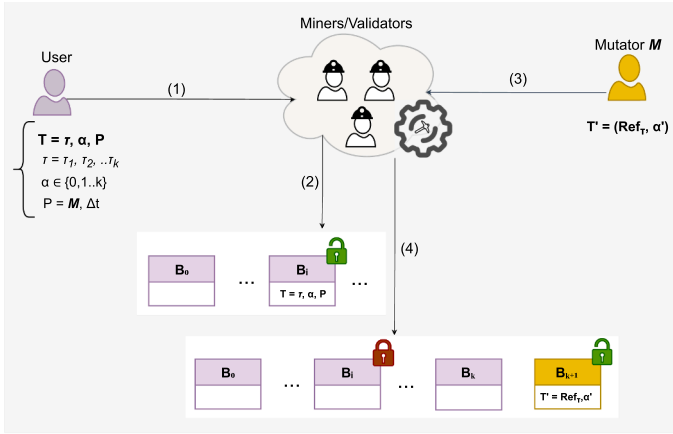


Fig. 5. μ chain redaction flow. (1) A user submits the mutable transaction $T = (r, a, P)$. (2) Miners validate the transactions and serve the decryption keys to the recipients. (3) A valid mutator M submits a mutant transaction T' to mutate T . (4) The miners verify and validate the mutation and add it to the chain, and stops serving the decryption keys for the old data.

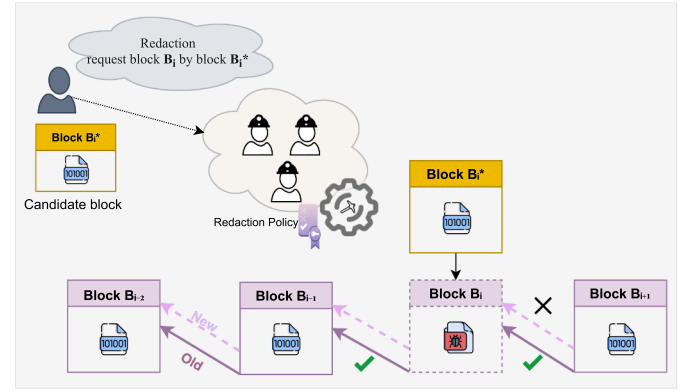
The *redacting transaction* also contains a proof of ownership created by hashing a secret known only to the user GVS and the previous transaction ID $P.TxID$, as well as their public key ($GV-PK^+$).

The network verifies this *redacting transaction* by checking that the revealed GVS and $P.TxID$ matches the stored GV in the original transaction, and that the signature is valid, proving that the user knows the corresponding PK^- to the $GV-PK^+$. Once validated, *Redacting transactions* are queued and processed in batches during a designated cleaning period to reduce overhead compared to per-transaction processing.

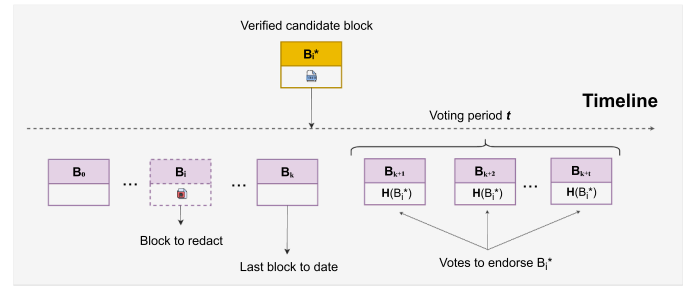
MOF-BC ensures that block integrity remains valid after data removal. In effect, block hashes are computed over transaction hashes and not their content, which allows removal without breaking hash consistency. Thus, when a transaction content is removed, its original hash remains in the block header, so the chain of block hashes is valid.

MOF-BC implements a dynamic fee structure based on the storage duration and size of transactions. Users are incentivized to optimize memory usage through rewards, encouraging behaviors like data summarization and timely deletion. The framework supports flexible Memory Optimization Modes (MOMs), such as temporary storage and summarization, reducing storage overhead by up to 25% compared to conventional blockchains [80]. On the other hand, While MOM flags and GV fields add flexibility, they also enlarge transaction size and impose extra processing overhead on miners and agents to detect and execute redaction meta-transactions. Similarly, the batch processing delay (Cleaning Period) means redactions are not immediate, which may not suit real-time applications. Furthermore, MOF-BC's model confines redaction authority strictly to transaction owners, which limits redaction scope in some legal or regulatory scenarios.

Summary. Meta-transaction-based approaches shift the redaction paradigm from cryptographic rewriting to semantic reinterpretation. Instead of modifying existing data, they introduce auxiliary transactions that overwrite, mask, or extend historical records, while preserving the original data on-chain in encrypted or inactive form. These methods offer strong auditability, lightweight mutability, and low protocol intrusion, making them attractive for blockchains where historical data should be hidden but not deleted (e.g., forensics, compliance reversibility). They are generally easier to deploy in permissioned or resource-constrained environments. However, meta-transaction methods retain the original data, making them vulnerable to privacy leaks and storage inefficiencies. Their reliance on encryption and key secrecy also introduces key distribution and revocation challenges, and redaction granularity is often limited to transaction-level rather than field-level edits.



(a) When a user sends a redaction request, the miners first verify the candidate block against the redaction policy, then verify the new and old links with the adjacent blocks



(b) The voting process

Fig. 6. Voting-based redactable blockchain.

3.4. Consensus-based redaction

Voting-based redaction represents a distinct class of approaches that address the redaction problem by operating directly on the second pillar of blockchain immutability: consensus. Instead of entrusting redaction authority to individual validators through key-based mechanisms, these systems introduce collaborative agreement through explicit voting protocols. Participants evaluate redaction proposals and collectively approve or reject them based on predefined criteria such as quorum thresholds, policy compliance, or weighted stake. This section examines notable solutions that take advantage of the voting-based redaction approach, highlighting their benefits and implications in mutable blockchain networks.

Deuber et al. [81] proposed the first voting-based algorithm for block redaction adapted for permissionless settings regardless of the deployed consensus protocols. The proposal leverages dual link to implement data modification so that two hash links are retained between adjacent blocks. The initial link connects the blocks as they were initially created and mined, while the correction link refers to the modified new data at the block granularity, as shown in Fig. 6.

Users issue redaction requests to update the content of a block in the chain by proposing a candidate block with new data and submitting it for validation. A set of validators check the compliance of the proposed block with the redaction policy. The redaction policy essentially outlines the level of agreement under which data can be redacted. The validators initiate an on-chain voting process to vote for valid candidate blocks within the voting window specified in the redaction policy. The policy sets the vote threshold and specifies the duration of the voting process, which is often measured in terms of the number of blocks mined after the redaction request. The voting threshold indicates the number of votes needed from the miners in order for the redaction to proceed. If the redaction proposal receives a majority of votes that surpasses the specified voting threshold, it is considered approved. In this

case, the old block containing the data to be redacted is replaced by the updated version reflecting the approved changes. This updated block is then broadcasted to the network, ensuring that all participants have access to the modified data.

Technically, this model adds a field y_i to the block B_i to store the initial state of the original block data, $B_i = \langle s_i, x_i, ctr_i, y_i \rangle$ such that $s_i = H(ctr_{i-1}, G(s_{i-1}, x_{i-1}), y_{i-1})$ is the hash of the previous block and $y_i = G(s_i, x_i)$ contains the initial hash of the block B_i upon its creation and addition to the blockchain.

Any user can propose an edit for a block B_i by creating a new candidate block $B_i^* = \langle s_i, x_i^*, ctr_i, y_i \rangle$. The candidate block B_i^* is then submitted to the validators for verification. First, it is verified against the redaction policy. If valid, an on-chain voting mechanism is launched; the nodes vote for the candidate block by adding its hash $H(B_i^*)$ to the data x of the next block as such $x || H(B_i^*)$.

If the number of votes exceeds the threshold ρ specified in the policy, the network can proceed with the redaction. Upon redaction i.e., The block data x_i is modified to the new data x_i^* , the hash link between B_i and B_{i+1} is compromised. However, since the original copy of the block hash kept in y_i is intact, the original hash will still hold for verification.

When such a situation is encountered, the validators employ several steps to ensure the validity of the redaction. First, validators check the validity of the data in the proposed block. Second, they verify if the block has solved the proof of work (w.r.t. old links). Third, if the new link introduced by the edit is broken, validators further validate the integrity of the old links in the blockchain. Finally, validators check whether the proposed edit complies with the predefined policy to proceed in the redaction process. If these aforementioned conditions are met, it confirms the authentication and validity of the proposed block edits. In such scenario, the modifications are considered authorized and accepted within the blockchain system. However, if any of the conditions are not satisfied, this implies that the block has been tampered with without proper authorization.

In summary, the redaction method presented in [81] offers a reliable and accountable way to redact blocks in permissionless blockchain systems. However, the voting period required for approving valid redactions must be sufficiently long to ensure honest redaction approval, leading to poor scalability and low efficiency. Moreover, the alteration of the block structure renders this redaction method incompatible with standard blockchain models.

Building on Deuber's framework, Marsalek et al. [82] propose a blockchain architecture with two coordinated chains, a *Standard Chain* preserving the canonical transaction history, and a *Redaction Chain* maintaining tamper-proof logs of authorized data modifications. Similar to the governance model in Deuber's framework, redactions in Marsalek et al.'s system are incorporated only after a majority vote by authorized participants. The redaction itself is recorded as a redaction block appended to the Redaction Chain, ensuring that the history of modifications is publicly visible and auditable. A key contribution of Marsalek et al. is the support for arbitrary data redactions, extending beyond OP_RETURN fields or metadata.

Another solution was proposed by Xu et al. [83] proposes redaction of data in permissionless Proof-of-Stake (PoS) blockchain that achieves fast confirmation. The system leverages the PoS slot structure for both block production and redaction voting, allowing edits to be proposed, voted on, and confirmed within a single block in synchronous networks. A committee of validators, selected using Verifiable Random Functions (VRFs) proportionally to their stake vote on redaction proposals. The slot leader, chosen randomly based on stake for the assigned time slot, collects the redaction votes from the committee and aggregates them into a single multi-signature (msig), which serves as cryptographic proof that the quorum of validators approved the redaction. Both the msig and VRF proofs are recorded on-chain, and once the quorum threshold is reached, the redaction becomes final.

Li et al. [84] propose a redactable blockchain protocol that achieves instant redaction confirmation by decoupling redaction voting from the

consensus protocol. Their core idea is to have an external voting committee, selected independently, approve or reject the redaction proposal. The voting committee is instantiated using verifiable random functions (VRFs) in Proof-of-Stake (PoS) environments or computational puzzles in Proof-of-Work (PoW) settings. The voters generate signed votes for the proposal and broadcast them to their peers. Votes are not immediately included on-chain; instead, a leader aggregates votes that meet the quorum threshold. The leader constructs a redaction certificate including a Merkle tree of the votes, the list of signed approvals, and hash commitment to the original and redacted blocks. The certificate is embedded in the metadata of the next block, marking the redaction as final and allowing all nodes to verify it on-chain.

Thyagarajan et al. have proposed Reparo [85], a distinct redaction and repair mechanism that operates as a modular overlay on top of existing blockchain protocols. Reparo offers a distinct advantage by performing repairs on pre-existent data with no need to modify the underlying block structure or consensus algorithm. In concrete, any user can submit a repair proposal for a block which is subsequently deliberated on by a dynamically selected group of validators (also known as deciders). Repair proposals are in the form $rp := \langle pt, x', sp \rangle$, with pt being the block pointer, x' the new data, and sp the state update specifications (from st to st').

Miners are responsible for collecting repair requests and voting on them through a decision protocol (called deliberation). A miner mines the repair request (like other transactions) if it finds it worth discussion, then the deliberations ensue off-chain. They evaluate whether the proposal complies with the repair policy that dictates the constraints and requirements for repair operations and then they vote according to defined voting periods in the header of the new block. After the deliberation phase is finished, miners redact the original data and update the state sp in the next block. The new block includes w in its metadata, enabling global verifiability.

Meng et al. [87] extend Deuber's voting model by introducing a conflict-resilient redactable blockchain architecture using Byzantine agreement consensus. This work aims to correct the critical limitation of Deuber's model, namely, the extended redaction latency and the lack of mechanisms to solve conflicting redactions of the same block. The work of [87] suggests selecting a conflict-free subset of requests per round to remove conflicting redactions. Then it uses committee-based consensus (inspired by Algorand [89]) to reach agreement on a set of redactions, providing instant finality after the BA round terminates. This method also decouples data redaction from new block proposals. The system continues to append blocks while redaction rounds can be held in parallel. Crucially, Meng et al. [87] handles up to one third of Byzantine miners in the network, even when the adversaries adaptively corrupt users or flood conflicting redactions.

AerChain [86] introduces a redactable blockchain that operates fully anonymously in a permissionless PoW setting. Voter selection is done via a custom Proof-of-Work (cPoW), where miners generate redaction voting puzzles. Solving the puzzle grants the right to vote. The difficulty of the puzzle determines the voting weight. Voters sign their ballots using a *bLSAG* ring signature used in Monero [90], proving that they are legitimate while preserving voter anonymity. Nodes collect votes over a predefined voting period and then the votes are aggregated and counted. If a quorum is met, a redacted block or transaction is finalized by replacing B_i with B_i^* , and updating the list of old history storing prior Merkle roots for integrity. The network switches to the redacted view of the block. In particular, AerChain supports anonymity while granting public verifiability of redactions with low latency.

PRBFPT [91] introduces a voting-based redactable blockchain design combined with chameleon hash assistance, in which all nodes share a public trapdoor, and editing rights are granted through on-chain smart contract governance. Rather than obscuring or distributing the trapdoor, PRBFPT publishes it directly in the block header, enabling all nodes to compute chameleon hash collisions independently. Security is

Table 5
Comparison of voting-based redaction methods.

SOLUTION	MECHANISM	GRANULARITY	ADVERSARIAL MODEL	LATENCY	MAIN FEATURES
DEUBER et al. [81]	Dual-hash voting via block references	Block	$\leq 1/2$ adversarial miners	Slow	– Poor redaction efficiency – Limited to non-monetary redactions – No conflict resilience
MARSALEK et al. [82]	Dual-chain architecture (Standard + Correction)	Block	$\leq 1/2$ adversarial miners	Slow	– Partial monetary redactions supported – Heavy storage overhead – Compatible with legacy PoW chains
XU et al. [83]	Stake-weighted VRF-based voting	Block	$\leq 1/3$ corrupted stake; static adversary model	Fast	– Fast finality – Supports multiple redactions per block – Incentivized redactions
REPARO [85]	Off-chain deliberation + witness inclusion	Block, Tx, or state	Same as underlying consensus (PoW/PoS)	Moderate	– Repairability of existing contents – Policy-based governance – Works with unmodified chains – Supports state corrections and non-monetary redactions
LI et al. [84]	External VRF/PoW committee + redaction certificate	Block	$\leq 1/2$ computing power (PoW) or stake (PoS)	Fast	– Voting decoupled from consensus – Cryptographic redaction proofs – No built-in incentive layer
AERCHAIN [86]	Anonymous PoW-weighted voting (bLSAG + cPoW)	Block, Tx	$\leq 1/2$ computing power	Moderate	– Strong voter anonymity – Sybil resistance via PoW – Public verifiability via ring signatures
MENG et al. [87]	BFT committee agreement	Block	$\leq 1/3$ adaptive adversary	Fast	– Conflict-free redaction scheduling – Redactions decoupled from block extension – Secure against adaptive adversaries
DUAN et al. [88]	Committee agreement	Block	Same as underlying consensus (PoW/PoS/BFT)	Moderate	– Forced synchronization – Incentivized redactions – Hash link consistency.

preserved not through secrecy but through auditable voting and structural integrity constraints enforced at the protocol level.

All nodes share the same chameleon hash keys, including the trapdoor. Since redactions require consensus-backed authorization, PRBFPT treats the public trapdoor as non-threatening: it enables anyone to compute collisions, but only authorized edits are accepted during validation.

The redaction process in PRBFPT involves a smart contract-based locked voting scheme which determines whether a transaction should be edited or deleted. If the vote passes, all nodes execute a block insertion routine, creating a new block that replaces the offending data while preserving chain continuity. Importantly, chameleon hash collisions are used to maintain Merkle root and header hash consistency, so chain connectivity is preserved despite edits.

The dynamic membership of permissionless blockchains complicates the monitoring and synchronization of redactions. Chainknot [88] addresses this challenge through forced synchronization. Redaction proposals are submitted as transactions and evaluated by a dynamically formed voting committee derived from the underlying consensus mechanism. Upon approval, a designated redactor generates a new block containing the redacted content; this block becomes the new chain tip, and includes incentives for voters and the redactor. Crucially, nodes that refuse to adopt the redacted block lose the ability to verify or extend the chain. This approach enforces global convergence through incentivized consensus, unlike prior voluntary adoption schemes.

Table 5 provides a structured comparison of representative voting-based redaction protocols, highlighting their core mechanisms, redaction granularity, adversarial assumptions, latency characteristics, and distinctive design features.

Overall, voting-based redaction systems are characterized by several key properties. Redactions are user-initiated and then approved via consensus, involving a set of designated voters. These systems are generally coupled to the underlying consensus protocol, leveraging its security guarantees while introducing controlled mutability. Confirmed redac-

tions are accompanied by cryptographic evidence, such as aggregated signatures, verifiable certificates, or witness proofs, supporting public verifiability and auditability of changes. While the majority operate at block-level granularity, only a few, notably Reparo [85] and AeRChain [86], support transaction- or state-level precision. Backward compatibility remains limited, only Reparo [85] stands apart as a backward-compatible layer for seamless integration with existing blockchain systems.

Summary. Consensus-based redaction schemes redefine immutability as a collective social contract rather than a purely cryptographic constraint. These methods embed redaction logic into the consensus layer, requiring a majority of network participants to approve and validate modification proposals through on-chain or off-chain voting. This class shines in decentralized environments, offering transparent, accountable, and scalable redaction mechanisms well-suited for public blockchains. By integrating redaction into the consensus mechanism, these schemes avoid trapdoor reliance and allow open participation, often with support for proposal justification and public auditing. However, consensus-based redaction faces key challenges. Namely, latency due to multi-phase voting rounds, security risks from vote manipulation or Sybil attacks, and the difficulty in maintaining semantic consistency after redactions (especially for smart contracts or interlinked transactions). These systems are best suited for governance-driven environments where redaction decisions must be socially negotiated, and they represent an evolution towards self-regulating mutable blockchains.

4. Experimental evaluation

We adopt the Bitcoin framework to implement the core functionalities of the evaluated redactable blockchains, with Bitcoin serving as our baseline. This approach allows us to establish a benchmark that objectively compares the performance of the redactable blockchain systems with the well-established Bitcoin framework. To this end, we

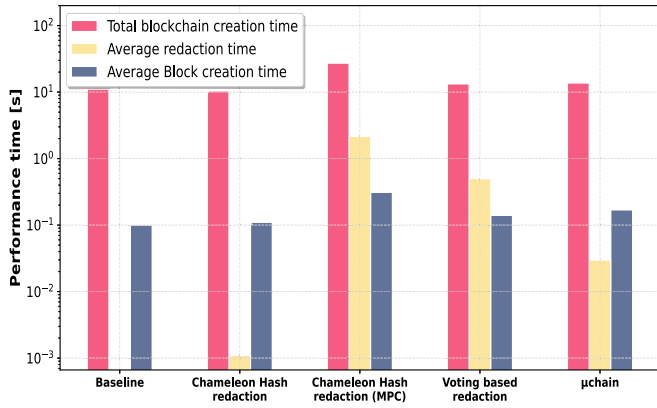


Fig. 7. Blockchain creation and redaction time comparison for a network of 1000 nodes (3% miners) processing 25k transactions under different redaction approaches. The permissioned chameleon hash-based approach achieves efficient redaction compared to voting-based and mutation-based approaches while incurring negligible overhead in block creation.

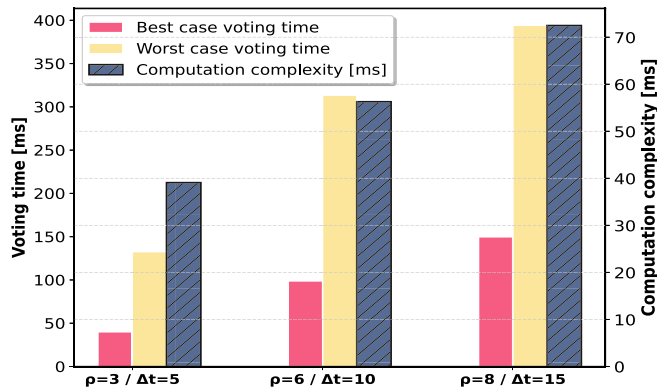


Fig. 8. Best and worst-case voting period complexity versus actual redaction time in voting-based redaction for varying voting periods Δt and vote thresholds ρ , evaluated on a network of 1000 nodes (3% validators) with 25k transactions. Higher ρ improves robustness but significantly increases redaction latency due to the distributed nature of voting.

implement three pioneering redaction solutions representing each respective classes (cryptographic [44], meta-transactions [78] and voting-based [81] redaction). We developed a custom discrete-event simulation framework. To the best of our knowledge, this represents the first open-source benchmarking suite specifically tailored for redactable blockchain systems. The framework is designed with modularity in mind, allowing the community to extend it with new redaction algorithms (e.g., zk-SNARK based schemes, VDF methods) by implementing standard interfaces for transaction processing and block validation.

Hardware Environment. All experiments were conducted on a physical workstation running Ubuntu 20.04 LTS. The machine was equipped with an Intel Xeon W-2123 Quad-Core processor (3.6 GHz base frequency) and 32 GB of DDR4 memory. All simulations were executed in a single-node environment to ensure deterministic control over network behavior and timing. The simulator extends the architecture of BlockSim, implemented in Python 3.10. It abstracts the network layer to eliminate transient internet fluctuations while preserving the logic of block propagation and verification. We prioritized a modular architecture to facilitate extensibility: cryptographic primitives are encapsulated in separate modules (e.g., a distinct Chameleon Hash interface), allowing researchers to seamlessly switch between different hashing methods or trapdoor implementations without refactoring the core consensus logic. The experimental framework builds upon and extends our system for simulating redactable blockchains proposed in [92]. For the

Table 6

Simulation parameters and network configuration.

PARAMETERS	VALUE
Simulation time	500000 s
Network size	1000 nodes
Miners fraction	30%
Simulation runs	10
Block interval	600 s
Block propagation delay	0.42 s

current work, we have significantly enhanced the framework to capture a broader range of system metrics, including throughput, latency, and energy consumption, while improving the scalability and configurability of the simulation environment. Our experimental framework yields valuable insight that may be extended to cover a wide spectrum of redactable blockchain systems. The source code, detailed configuration files, and raw data logs are publicly available at the GitHub repository: <https://github.com/ImaneElabid/Redactable-blockchains>

We evaluate the performance of the redactable blockchain system by conducting experiments in a controlled environment without malicious nodes. Unless stated otherwise, we follow this setup: over a simulated period of 500,000 seconds (discrete-event time), we construct a blockchain within a network of 1000 nodes with 30% of miners. The results are averaged over 10 simulation runs for more precision. To approximate real-world conditions, we integrate system parameters such as block intervals and propagation delays derived from the Bitcoin network model, as summarized in Table 6.

Redaction time overhead. Fig. 7 shows the performance time of the three redaction methods benchmarked against a standard Bitcoin model.

From a blockchain creation and block generation perspective, the results reveal remarkably consistent performance across all redaction techniques in comparison with the baseline. The results indicate that the choice of redaction mechanism has minimal impact on the fundamental block creation process. The most notable observation is that chameleon hash with MPC shows slightly higher creation time, likely due to the additional computational overhead of multi-party computation protocols during the overall blockchain construction process. The differences are more pronounced for the redaction time. The permissioned Chameleon Hash method demonstrates near-instantaneous redaction, facilitated by a single authorized entity. In its permissionless (MPC) variant, where trapdoors are secret-shared among multiple parties, the redaction time increases significantly. Voting-based redaction remains approximately 100× slower than centralized chameleon hash. This is a direct result of the extended voting periods measured in terms of the mined blocks following the redaction initiation (in this experiment, the voting period Δt is set to 10 blocks and the vote threshold ρ is 6 votes). The μ chain's redaction time is moderately higher than the Chameleon Hash method, reflecting the additional steps of policy verification and transaction replacement within its structure. Notice that the scale is logarithmic.

Fig. 8 further investigates the impact of the voting period in the voting-based scheme on redaction time complexity including both the verification time and the voting time, which determines the computational complexity of the redaction. Fig. 8 showcases the best and worst case time for a redaction operation. The best case scenario involves collecting votes from the ρ blocks immediately following the candidate block verification. While the worst case scenario requires collecting votes by the end of the voting period. Notably, the gap between computation complexity and voting time is significant because voting is a distributed task performed by validators, while other miners continue to create and propagate new blocks.

Exploring the extremely low performance of chameleon hash with MPC in the first experiment, Fig. 9 reveals that the redaction exhibits exponential time complexity with the number of validators, via two

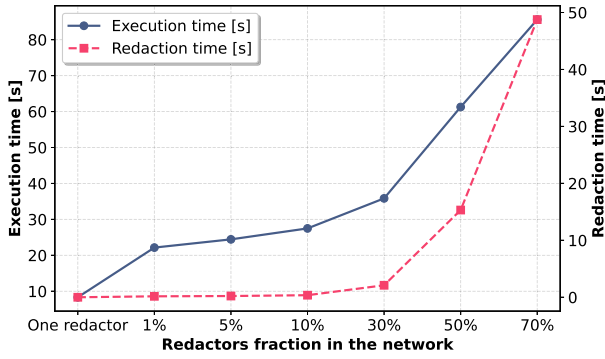


Fig. 9. Impact of the number of validators on redaction time in a chameleon hash-based redaction system with secret key sharing, on a network of 1000 nodes. Redaction time remains stable with up to 30% validator participation.

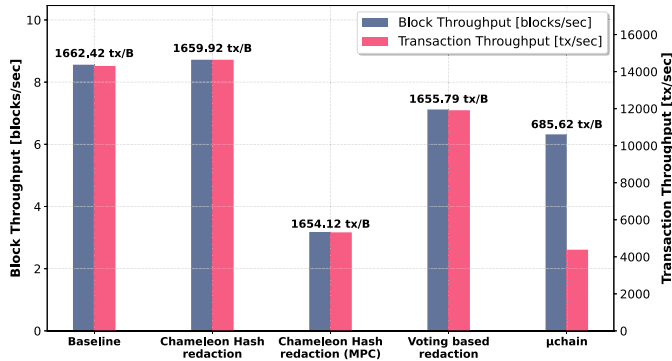


Fig. 10. Transaction and block throughput comparison across redaction approaches processing 25k transactions. Permissioned chameleon hash achieves the highest throughput. Voting-based redaction shows a slight throughput reduction, while μ chain exhibits a significant drop in transaction throughput.

distinct performance phases. In the initial phase (approximately 0 to 30% validator fraction) the redaction time remains relatively stable at around 0.5 seconds, suggesting that the secret sharing reconstruction overhead is manageable when only a subset of validators participate. The flat region indicates that the system can efficiently handle moderate validator participation without significant performance decay. However, beyond 30% threshold, the complexity transitions into an exponential growth, where redaction time increases dramatically as validator participation approaches 100% of the network size. Consequently, distributed chameleon hash schemes impose substantial performance penalties that become prohibitive as network size increases.

Redaction throughput and storage overhead. In the second set of experiments, we assess the ratio of effective transactions among the evaluated redactable blockchain systems. Specifically, we measure the average number of transactions per block, transaction and block throughput, and overall storage efficiency.

Our results in Fig. 10 indicate a slight reduction in block throughput compared to the baseline for chameleon hash and voting approaches, while they maintain a similar number of transactions per block. Chameleon hash with MPC exhibits a dramatic throughput reduction, indicating that the MPC overhead primarily affects block production rate. The MPC, though, keeps the same transaction density per block.

Most notably, μ chain shows a substantial decrease in transactions per block with approximately 60% fewer transactions than other methods, while achieving comparable block throughput. To validate this observation, Fig. 11 confirms that the chameleon hash and voting solutions maintain the same number of transactions per block as the baseline, whereas the μ chain with different versioning configurations (μ chain

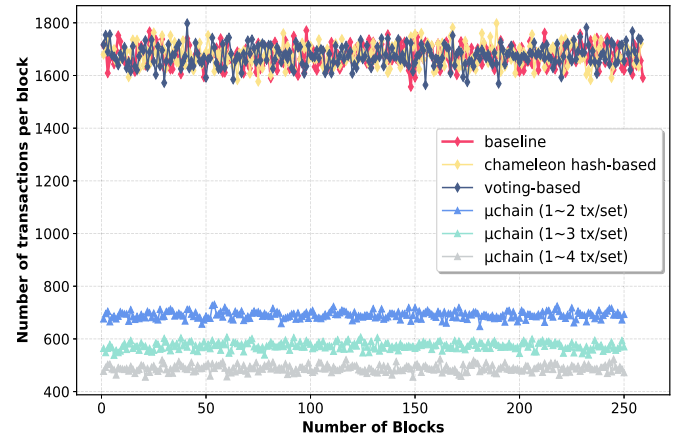


Fig. 11. Transaction counts per block in different redaction techniques. The μ chain configurations (e.g., 1~2 tx/set) indicate each set contains 1 to 2 transaction versions. A fixed “nope” transaction is included with every mutable transaction set, contributing to the overhead.

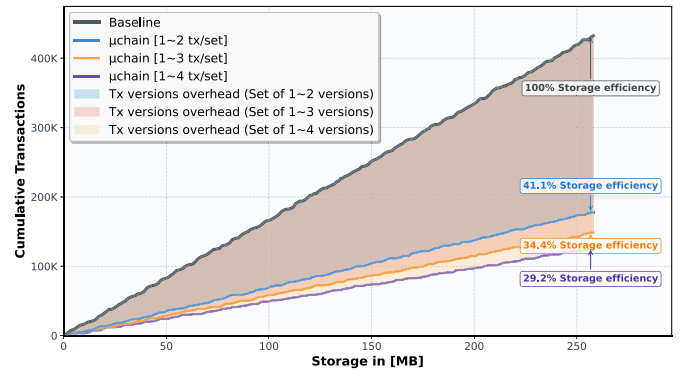


Fig. 12. Cumulative transaction counts versus storage growth between the baseline and μ chain configurations. The colored regions between the baseline (100% efficient) and μ chain configurations (41.1%, 34.4%, and 29.2% efficient for 1~2, 1~3, and 1~4 version sets respectively) represent the wasted storage capacity dedicated to transaction versioning.

with 1~2, 1~3, and 1~4 tx/set configurations) has a substantially lower number of transactions per block.

The throughput penalty of the mutation-based solution is further compounded with a severe storage inefficiency, as demonstrated by Fig. 12. The figure plots the cumulative number of transactions against the total storage used, revealing that a large portion of the allocated storage is wasted on maintaining transaction versions rather than storing new, unique transactions. Noticeably, even the μ chain [1~2 tx/set] configuration achieves only 41.1% storage efficiency, meaning nearly 60% of its allocated storage is occupied by versioning overhead rather than new transactions. Higher versioning configurations perform even worse, incurring a substantial and inherent storage penalty as the versions increase.

To examine the size overhead induced by the modifications to the block structure to build redactable blockchain systems. Table 7 shows the new block fields sizes required in each of the three redaction approaches and their respective impacts on the total size overhead. Table 7 indicates the additional storage space required to implement these modifications compared to the baseline Bitcoin data structure. The size overhead is defined as the additional load in the header and data field, as well as the overall impact on the entire block. Chameleon hash-based approach increases the size of the header by 40% due to the addition of hash randomness, but this results in only 0.0112% increase in the size of the entire block. In the case of the Voting-based approach, an extra field refers to the initial block state field y is included in the

Table 7
Size overhead comparison to the baseline.

COMPONENT	ATENIESE et al. [20]	DEUBER et al. [81]	PUDDU et al. [78]
Header	Hash randomness r (+40%)	Initial block state y (+40%)	–
Data	–	Votes (+0.06%)	Transaction set T (+71%)
Total	+0.0112%	+0.0711%	+71%

block header, resulting in a size overhead increase of approximately 40% of the header size. Additionally, votes are added to the data field, which leads to a negligible increase of 0.06% of the data size, resulting in a total size overhead of 0.0711%. In contrast, μ chain has the largest size overhead of a 71% increase in the size of the data field due to the addition of transaction versions.

Our findings regarding block size variations are further supported by a granular analysis of the storage complexity across the evaluated systems. Table 7 quantifies the new block fields sizes required by each of the three redaction approaches and their resulting storage overhead. Table 7 indicates the additional storage required to implement these modifications compared to the Bitcoin model. The size overhead is defined as the additional load in the header and data field, as well as the overall impact on the entire block.

Redaction energy overhead. In this set of experiments, we evaluate the energy consumption of the redaction approaches relative to the baseline. Fig. 13 reports the average energy consumption (in joules) for each approach when performing five redactions. Permissioned chameleon hash-based redaction exhibits the lowest overhead, consuming less than twice the baseline energy within a short redaction time. In contrast, the permissionless variant (MPC) requires more than 8 $\times$ the baseline energy. For the voting redaction, although the redaction time is eight times longer than that of permissioned chameleon hash, the energy consumption is less than 1.2 $\times$ higher, since most of the period is spent on lightweight voting rather than heavy computation. By comparison, μ Chain consumes about 1.6 $\times$ more energy than permissioned chameleon hash, as redaction involves updating the block to select and set the appropriate transaction version as default.

Features and trade-offs. In line with our experimental analysis, we compile a summary comparing the key features, advantages, and drawbacks of the aforementioned redaction approaches (i.e., chameleon hash-based, voting-based, and mutation-based redaction). Table 8 systematically examines the primary redaction solutions in each approach, covering aspects such as network settings, granularity, storage overhead, among others. The synthesis presented in Table 8 serves as a starting point to establish a well-informed selection and a better design of redactable blockchain systems that effectively balance the trade-offs between redaction capabilities, storage efficiency, and security considerations.

In the chameleon hash-based approach represented by the seminal work of Ateniese et al. [20], we observe minimal redaction time due to the intervention of a central entity responsible for the redaction process. However, from a security standpoint, centralization has always been linked to single point of failure attacks, which directly contradicts the fundamental principle of decentralization in blockchain systems.

More concretely, the chameleon hash approach [20], makes use of two major mechanisms to perform a reduction on block-level; chameleon hash and a secret sharing scheme for key distribution. A secure MPC protocol is deployed in case of a consortium or a public network to secretly share the trapdoor key among the validators. This technique can be inefficient when the number of players involved in the MPC protocols is big, as it leads to reduced performance.

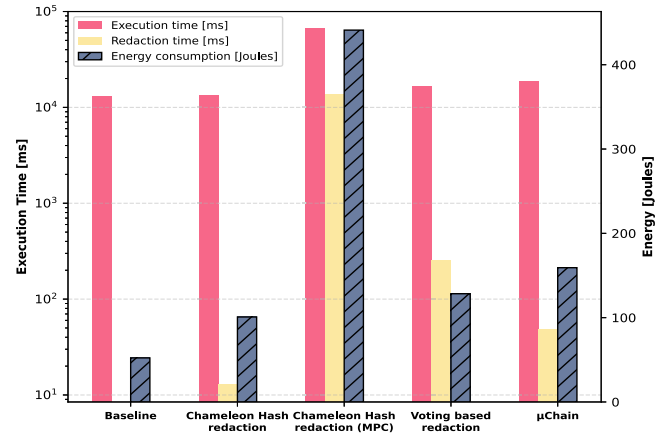


Fig. 13. Average energy consumption and execution time of different redaction approaches relative to the baseline. Permissioned chameleon hash-based redaction is the most efficient, while permissionless chameleon hash with MPC incurs significantly higher energy consumption compared to other approaches.

Additionally, chameleon hash-based techniques offer low storage overhead but may face challenges related to decentralization, key management. Besides key management issues embodied in the chameleon hash solution, a notable limitation of this approach is the lack of consideration for transactions dependencies when performing redactions. This makes the system unsuitable for monetary transactions, as only stateless redactions are considered. All these elements makes this approach to be not deployable in existing blockchain systems.

The redaction algorithm of Ateniese et al. [20] considers that immutability property of blockchain records are the norm, and that redaction operations are supposed to happen very rarely. This implies that editors do not operate ceaselessly, but only in exceptional circumstances. Besides, chameleon hash based redaction mechanisms are not compatible with existing blockchains. Not only do they require modifications to the block header structure, but they also do not interoperate with the currently deployed hash functions (e.g. SHA256) and require the use of chameleon hash instead.

On the other hand, mutation-based methods involve complex operations such as multiversions, multi-key encryption, and secure multiparty computation. These methods offer transaction consistency but incur high storage overhead due to the need for versioning and encryption. Challenges in mutation-based approaches include key management, storage efficiency, and ensuring backward compatibility.

μ chain [78] is also a solution for private blockchain. This solution enables mutations on a transaction level by leveraging multi-versions of transactions with a multi-key encryption to indicate the active transactions. By adding the mutated versions in next blocks, the old data is kept on-chain with no possibility to shrink the chain which result in a storage overhead.

For permissionless networks, a secret sharing scheme is set up to distribute the encryption keys in networks with no trust assumptions over the validators. The usage of secret sharing schemes for key management involves MPC which leads to additional computational cost in the system. Besides, the modifications in the data structure makes the μ chain unable to ensure backward compatibility. Although, μ chain considers the dependent transactions of a mutated transaction to correct the missing amount of cryptocurrency in the account. Other weaknesses need to be reviewed, especially related to policy. Since the policy is set by the sender itself, malicious users can submit illicit content without giving the mutators the right to change it.

As the previous solutions rely on additional trust assumptions, ranging from secret sharing to the MPC protocol, to distribute the keys associated to transactions encryption. These solutions clearly fall short of the requirements of permissionless blockchain models.

Table 8
Comparison of redactable blockchain techniques.

CATEGORY	CRYPTOGRAPHIC-BASED	VOTING-BASED	MUTATION-BASED
CORE MECHANISMS	CH, ZKP, RSA, Ring Signatures, MPC	Dual Hash Links, On-chain Voting, Logs	Multi-versions, Encrypted versions, Dependency tracking
REDACTION AUTHORITY	Designated redactors	Network quorum via on-chain votes	Off-chain approval with enforced consistency
GOVERNANCE MODEL	Centralized trusted setup or Distributed (e.g., DKG, TUCH)	Transparent, verifiable voting procedures	Hybrid trust: off-chain control, on-chain proof
GRANULARITY	Block, transaction, or transaction-field level	Block-level only	Transaction-field level
CONSISTENCY GUARANTEE	Collision validity, State consistency	Voting confirmation and chain linking	Dependency-aware revalidation
LATENCY	Near-instantaneous (1 block)	High latency (multi-block)	Moderate, depends on propagation
STORAGE OVERHEAD	Low to moderate: trapdoor, ZKP, proofs	Moderate: vote logs, dual headers	High: versioning, diffs, logs
COMPUTATION OVERHEAD	Trapdoor ops, key and proof generation	Vote collection and quorum checks	Diff tracking, encryption/decryption
AUDITABILITY	Limited (trust-based assumptions)	High (votes are public)	Moderate (log-based)
KEY EXPOSURE RISK	High (persistent across schemes)	None (no keys)	Moderate (encryption keys)
OLD DATA DELETION	Obfuscated/rewritten	Marked via votes	Encrypted/rolled back
BEST-FIT APPLICATIONS	IoT, constrained devices, industry blockchains	Public ledgers, community governance	GDPR systems, archival systems
MAIN STRENGTHS	Fast edits, bounded trust, lightweight	Accountability, verifiability	Semantic edits, strong consistency
KEY LIMITATIONS	Key setup/management, weak public traceability	High latency, scalability issues	Heavy storage, key management, complex updates

As for voting-based approaches, they exhibit longer redaction time compared to block creation due to the need for extended voting periods and consensus among network participants. However, these methods provide a mechanism for public verification and allow for content analysis during the redaction process. They also introduce considerations such as voting period length, redaction efficiency, incentives, and the ability to handle monetary and non-monetary content. The voting-based redaction, as presented in several studies [81–85] enables public verifiability by preserving hash links to both the original and modified data. This feature permits any node in the network to verify the accuracy of the redactions and the data. Except for Reparo [85], all other voting-based redaction solutions lack backward compatibility with existing blockchains, as they modify the data structure of the blocks. o, in the other hand, stands out for operating on top of the blockchain structure, facilitating the integration with any existing blockchain. Furthermore, most systems impose restrictions on the types of data to be redacted. Redaction is restricted to non-spendable data to avoid consistencies and conflicts in payment systems.

5. Discussion

Redactable blockchains have garnered interest for their versatile qualities, however many important questions regarding their scalability, deployment, and performance remain unresolved. This discussion highlights critical research questions, including scalability, consistency, and adaptability among others. We believe that addressing these questions is critical for a large-scale adoption of redactable blockchain solutions in real-world applications.

5.1. Governance illegibility and privilege abuse

A key governance challenge in redactable blockchains is the opacity of redaction privileges. Proposals vary from unilateral redactors with full cryptographic control to decentralized voting. For instance, Ateniese's foundational chameleon hash [20] assumes rare, trusted

edits by a trapdoor holder but lacks visibility into frequency, intent, or misuse. Many similar systems rely on unverifiable edits and honest-majority validators, exposing them to collusion. Policy-based solutions [46,49,62,63] improve transparency via access control and traitor tracing yet fail to establish penalty structures or reputation tracking for malicious validators.

Permissioned models [51,55,59] assign redaction authority to regulators or privileged nodes under fixed roles. While suitable for regulatory compliance (e.g., GDPR), this model introduces centralized trust dependencies. Open voting models [81,83,85] risk spam and abuse, while owner-only mutation models [78,80] allow selective deletion of sensitive records. Systems like RedactChain [66] and μ Chain [78] introduce redaction windows, aligning with data minimization but limiting long-term erasure rights under GDPR.

In summary, while regulatory compliance motivates many redactable blockchain designs, most systems often prioritize cryptographic techniques over governance transparency. Future systems must integrate explicit privilege assignment, verifiable accountability, and robust enforcement within frameworks that meet legal standards.

5.2. Cost vs. benefit balance

Redaction operations introduce tangible overheads that conflict with profit-maximizing block production. Without explicit incentives, rational validators may ignore or delay redactions to preserve their mining advantage. Hence, a pay-for-performance strategy is essential to incentivize validator cooperation and ensure redacted data integrity. While some proposals incorporate basic incentive models, most lack formal reward-aligned mechanisms. For example, Shen et al. [57] reward redaction approvals and penalize misbehavior. Similarly, CDEdit [59] requires users to buy tokens for redactions, compensating redactors accordingly. In voting-based schemes like Reparo [85] or [81], redaction fees exist but typically reward block proposers, not voters, leaving redactors uncompensated. This discourages validator engagement. Furthermore, systems with low-cost redaction requests are vulnerable to

DoS and Sybil attacks, as adversaries may flood the network with malicious proposals. Xu et al. [83] introduces a incentive mechanism where validator rewards are weighted by stake, but still lacks penalties for non-participation.

In summary, incentive design for redactable blockchains remains a critical gap. Without mechanisms to compensate validators fairly and deter abuse, redaction systems risk limited adoption in permissionless environments where spontaneous participation cannot be assumed.

5.3. Conflicts resolution and inconsistencies

Another flaw in the design of redactable blockchains is the potential inconsistencies in the ledger that occur after redactions are made. In particular, most systems lack mechanisms to resolve conflicting edit requests that target the same data with different objectives or granularities. When multiple parties initiate overlapping redactions, existing designs typically serialize requests or reject conflicts outright, rather than resolving them semantically. For example, in intellectual property or provenance tracking, one party may request the removal of a record while another simultaneously asserts partial ownership. Without conflict-aware resolution logic, the ledger risks converging to ambiguous or contradictory states, undermining determinism and trust. Only a limited number of proposals explicitly address this issue. Wolverine [65] formalizes transaction dependencies and enforces consistency-preserving redactions by rejecting or cascading edits that would violate semantic validity, while Meng et al. [87] introduce conflict-free redaction scheduling using Byzantine agreement to ensure that only non-conflicting redaction sets are finalized per round.

Additionally, redactable blockchains often lack version control to manage consistency across nodes. In the absence of canonical redacted states, nodes may hold divergent block histories. “Zombie data” can persist on outdated chains, and nodes broadcasting stale versions degrade global consistency. Malicious actors may exploit this to reintroduce obsolete or sensitive data into the network. Some recent designs explicitly target post-redaction synchronization. Shen et al. [57] employ cryptographic accumulators to bind the active set of valid blocks, enabling nodes to detect superseded versions, while Wang et al. [70,71] propose VDF-based redactable blockchains that enforces strong synchronization of redaction, ensuring all nodes maintain identical chain views post-modification.

The consistency problem is further exacerbated in voting-based redaction schemes. As observed by Dousti et al. [73], an adversary can redact a block that contains a vote for a pending redaction, erasing part of the vote history. This violates verifiability for new nodes, which can no longer validate the original redaction due to missing votes.

In summary, redactions are often treated as local edits, with the implicit assumption that all nodes will adopt the redacted chain. However, redactions constitute state transitions requiring global consistency guarantees. Each modification triggers cascading state recomputations, induces multiple version branches, and requires cross-node consistency proofs for global convergence. Without explicit mechanisms for conflict resolution, authenticated version propagation, and tamper-resistant vote records, redactable blockchains remain vulnerable to divergence, rollback attacks, and unverifiable histories.

5.4. Redaction challenges in permissionless settings

A critical bottleneck lies in committee formation and rotation mechanisms. Schemes like NITCH-based approaches [65] demonstrate that non-interactive threshold chameleon hashes can eliminate coordination overhead during collision generation. However, the committee selection process itself introduces significant complexity. Current approaches rely on either stake-weighted selection (as in [50]) or computational puzzles (as in AeRChain [86]), but both face inherent trade-offs.

On the other hand, Frequent rotation (e.g., per-epoch as in Wolverine [65]) enhances security by limiting the window of trapdoor exposure

and mitigating long-term collusion risks. However, it introduces substantial on-chain overhead for Distributed Key Generation (DKG) protocols and increases the attack surface during transition periods when both old and new committees must coordinate. Conversely, infrequent rotation reduces overhead but extends the period during which a compromised committee can perform unauthorized redactions. Existing work provides limited empirical guidance on optimal rotation schedules, and no scheme has yet demonstrated a formal security-performance trade-off analysis that accounts for adversarial adaptive corruption strategies.

5.5. Scalability, latency, and the storage minimization

Redactable blockchains aim to remove sensitive or obsolete data, yet most existing solutions paradoxically introduce considerable overhead in computation, latency, and storage. Chameleon-hash-based schemes incur costs from trapdoor key management, DKG, MPC, and ABE enforcement. ZKP-based protocols [75,76] require intensive proof generation and verification. Lattice-based systems [60] add further load due to high-dimensional algebraic operations. Voting-based protocols [81,82,85] suffer from delayed finality caused by multi-round broadcasts and long voting periods, making them unsuitable for time-sensitive redactions under regulations like GDPR.

Many systems inflate ledger size through versioning, proofs, dependency tracking, and metadata. ReTRACe [58] and Wolverine [65] maintain redundant metadata, tamper-evident proofs, and revocation logs that persist after redactions. μ Chain [78] requires 71% heavier blocks to store multi-version transactions and encryption keys. Zero-knowledge proof-based schemes [75,76] generate proofs ranging from 40-60KB per redaction, which can bloat the blockchain substantially when multiplied across thousands of operations. While zk-SNARKs offer succinctness, their trusted setup ceremonies introduce centralization risks in permissionless settings. Voting schemes [81] introduce 40% header overhead for dual hashes and vote logs. On the other hand, MOF-BC [80] achieves 25% storage savings by summarizing outdated records. Low-latency protocols like Xu et al. [50] approve redactions within a single block, significantly reducing latency.

In summary, the substantial overheads in cryptographic processing, multi-round voting, and persistent metadata storage highlight the impracticality of uniform mutability guarantees across heterogeneous applications. Context-aware redaction schemes must adapt guarantees and optimizations to domain-specific requirements. Healthcare systems may tolerate overhead for regulatory auditability. IoT networks may enforce lightweight, time-bounded edits for efficiency. Central bank infrastructures might adopt post-quantum redactions where computational cost is secondary. Redactable blockchain design must align mutability semantics with application constraints while preserving distributed ledger trust properties.

5.6. Redaction in emerging blockchain architectures

Current proposals are outpaced by evolving architectures, suffering from dependence on pre-quantum assumptions and incompatibility with modular, layered execution models.

5.6.1. Post-quantum redactability

The cryptographic foundations of existing redactable blockchains, such as chameleon hashes based on discrete logarithm or RSA hardness [20,72], pairing-based attribute encryption [46], and elliptic curve zero-knowledge proofs [75], face serious compromise under quantum adversaries. While Wu et al. have pioneered lattice-based chameleon hashes and Botta et al. offer post-quantum ZK-proofs, these solutions often incur significant performance penalties. Preliminary lattice-based constructions [60] and post-quantum ZK schemes [75] represent isolated efforts to demonstrate theoretical viability in quantum-vulnerable environments. They also impose prohibitive overheads from large parameters and high computational overhead. Future research must

therefore move beyond proof-of-concept designs and focus on performance-oriented post-quantum primitives, such as compact lattice trapdoors or hash-based proof systems (e.g., STARKs) specifically optimized for blockchain redaction, ensuring that long-term immutability does not become long-term vulnerability.

5.6.2. L2 and cross-layer consistency

Redaction research remains predominantly bound to Layer-1 execution models, implicitly inheriting the throughput limits, latency constraints, and cost structures of base-layer consensus. Most existing designs implicitly assume that redactions are infrequent, high-impact events executed at settlement-layer cadence. In practice, compliance-driven mutability such as GDPR erasure, log sanitization, and application data updates occurs at frequencies incompatible with Layer-1 throughput and cost models. As user data and business logic increasingly migrate to Layer-2 systems, executing redactions directly on the base layer represents an architectural inefficiency. Layer-2 mechanisms such as optimistic and zk-Rollups could potentially batch redaction operations off-chain, posting only cryptographic commitments or validity proofs on-chain. Despite this opportunity, there is a notable absence of cross-layer redaction protocols that formally bind Layer-1 authorization to Layer-2 execution. While systems like Reparo [85] suggest that redaction can function as an overlay layer. This concept could be extended to an independent modular redaction protocol that sits between the consensus and execution layers, filtering or modifying state transitions before finalization. Such a design would allow legacy immutable chains to support regulatory compliance without hard forks, simply by upgrading the execution client to recognize the redaction overlay. Crucially, future architectures must enforce a clean separation of responsibilities, with Layer-1 should act as the canonical authority and policy ledger for redaction rights, while Layer-2 systems handle scalable execution and generate cryptographic evidence of compliant state transitions.

6. Conclusion

Redactable blockchain technology is a complex, multidimensional field in its nascent stage. Building on this premise, our exploration of blockchain redaction methods provides a critical perspective on this less-explored facet of blockchain technology. Our analysis and experimental comparison of various redaction methods, including chameleon hash-based, mutation-based, and voting-based techniques, have underscored the potential of redactable blockchain systems. Our study of this new paradigm is not just an exploration of the current state of redactable blockchain but also a compelling catalyst for future research. However, we acknowledge that the list of redaction methods discussed in this review paper is not exhaustive. The selection of redaction methods included in our study was based on their significance, representation of different approaches, and availability of research data. We anticipate that our insights would serve as a stepping stone for future research, providing a comprehensive overview of the current landscape and paving the way for the development of more robust, efficient, and secure redactable blockchain systems.

Data availability

No data was used for the research described in the article.

CRedit authorship contribution statement

Imane El Abid: Writing – review & editing, Writing – original draft, Software, Methodology, Formal analysis, Data curation, Conceptualization; **Karim Boubouh:** Writing – review & editing, Writing – original draft, Validation, Software; **Yahya Benkaouz:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] D. Mingxiao, M. Xiaofeng, Z. Zhe, W. Xiangwei, C. Qijun, A review on consensus algorithm of blockchain, in: IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, 2017.
- [2] S. Nakamoto, Bitcoin: a peer-to-peer electronic cash system, decentralized business review 2008, p. 21260.
- [3] V. Buterin, et al, A next-generation smart contract and decentralized application platform, White paper 337 (2014) 1–2.
- [4] U.W. Chohan, The double spending problem and cryptocurrencies, Available at SSRN 3090174 (2021).
- [5] B.-K. Zheng, L.-H. Zhu, M. Shen, F. Gao, C. Zhang, Y.-D. Li, J. Yang, Scalable and privacy-preserving data sharing based on blockchain, J. Comput. Sci. Technol. 33 (3) (2018) 557–567.
- [6] C. Yang, X. Chen, Y. Xiang, Blockchain-based publicly verifiable data deletion scheme for cloud storage, J. Netw. Comput. Appl. 103 (2018) 185–193.
- [7] E. Politou, F. Casino, E. Alepis, C. Patsakis, Blockchain mutability: challenges and proposed solutions, IEEE Trans. Emerg. Top Comput. 9 (4) (2019) 1972–1986.
- [8] D. Zhang, J. Le, X. Lei, T. Xiang, X. Liao, Exploring the redaction mechanisms of mutable blockchains: a comprehensive survey, Int. J. Intell. Syst. 36 (9) (2021) 5051–5084.
- [9] S.M. Ali, M.N. Yusoff, H.F. Hasan, Redactable blockchain: Comprehensive review, mechanisms, challenges, open issues and future research directions Future Internet 15 (2023) 35.
- [10] T. Ye, M. Luo, Y. Yang, K.-K.R. Choo, D. He, A survey on redactable blockchain: challenges and opportunities, IEEE Transactions on Network Science and Engineering 2023.
- [11] E. Fathalla, C. Wang, X. Li, R. Gazda, H. Wu, Redactable distributed ledgers: a survey, Distrib. Ledger Technol. Res. Pract. 2 (3) (2023) 1–26.
- [12] J.W. Heo, G.S. Ramachandran, A. Dorri, R. Jurdak, Blockchain data storage optimisations: a comprehensive survey, ACM Comput. Surv. 56 (7) (2024) 1–27.
- [13] 17 GDPR - right to erasure ('right to be forgotten', 2018. <https://gdpr.eu/article-17-right-to-be-forgotten/>
- [14] R. Matzutt, M. Henze, J.H. Ziegeldorf, J. Hiller, K. Wehrle, Thwarting unwanted blockchain content insertion, in: 2018 IEEE International Conference on Cloud Engineering (IC2E), IEEE, 2018, pp. 364–370.
- [15] Q. Feng, D. He, S. Zeadally, M.K. Khan, N. Kumar, A survey on privacy protection in blockchain system, J. Netw. Comput. Appl. 126 (2019) 45–58.
- [16] M.I. Mehar, C.L. Shier, A. Giambattista, E. Gong, G. Fletcher, R. Sanayhie, H.M. Kim, M. Laskowski, Understanding a revolutionary and flawed grand experiment in blockchain: the dao attack, J. Cases Inf. Technol. (JCIT) 21 (1) (2019) 19–32.
- [17] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. Mccorry, A. Gervais, SOK: off the chain transactions, IACR Cryptol. ePrint Arch, 2019, 2019, p. 360.
- [18] U. Pagallo, E. Bassi, M. Crepaldi, M. Durante, Chronicle of a clash foretold: Blockchains and the GDPR's right to erasure, Legal Knowledge and Information Systems, JURIX 2018 2018, 81–90.
- [19] L.-D.I. nez, K. O'hara, E. Simperl, On blockchains and the general data protection regulation, Technical Report, EU Blockchain Forum and Observatory, 2018.
- [20] G. Ateniese, B. Magri, D. Venturi, E. Andrade, Redactable blockchain-or-rewriting history in bitcoin and friends, in: 2017 IEEE European symposium on security and privacy (EuroS&P), IEEE, 2017.
- [21] N. Szabo, Formalizing and Securing Relationships on Public Networks, First Monday, 1997.
- [22] J. Garay, A. Kiayias, N. Leonardos, The bitcoin backbone protocol: analysis and applications, in: Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, 2015.
- [23] A. Gervais, G.O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, S. Capkun, On the security and performance of proof of work blockchains, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, the 2016 ACM SIGSAC Conference on Computer and Communications Security, 2016, pp. 3–16.
- [24] T. Rocket, Snowflake to avalanche: a novel metastable consensus protocol family for cryptocurrencies, 2018.
- [25] I.E. Abid, Y. Benkaouz, A. Khoumsi, LightX: a lightweight proof-of-bandwidth transactions transfer system, in: Networked Systems: 9th International Conference, NETYS 2021, Virtual Event, Springer, 2021, pp. 144–160.
- [26] R. Guerraoui, P. Kuznetsov, M. Monti, M. Pavlovic, D.-A. Seredinschi, At2: asynchronous trustworthy transfers, 2018. arXiv:1812.10844
- [27] I. Eyal, A.E. Gencer, E.G. Sirer, R.V. Renesse, BitCoin-NG: a scalable blockchain protocol, in: 13th USENIX Symposium on Networked Systems Design and Implementation, 2016, pp. 45–59.
- [28] M. Castro, B. Liskov, et al, Practical byzantine fault tolerance, OSDI 99 (1999) 173–186.
- [29] R. Zhang, R. Xue, L. Liu, Security and privacy on blockchain, ACM Comput. Sur. (CSUR) 52 (3) (2019) 1–34.
- [30] Y. Sompolinsky, A. Zohar, Secure high-rate transaction processing in bitcoin, in: Financial Cryptography and Data Security: 19th International Conference 19 of San Juan, Puerto Rico, Springer, FC 2015.

- [31] A. Narayanan, J. Bonneau, E. Felten, A. Miller, S. Goldfeder, Bitcoin and cryptocurrency technologies: a comprehensive introduction, Princeton University Press, 2016.
- [32] G. Wood, et al, Ethereum: a secure decentralised generalised transaction ledger, Ethereum Project Yellow Pap. 151 (2014) 1–32.
- [33] R. Matzutt, J. Hiller, M. Henze, J.H. Ziegeldorf, D. Müllmann, O. Hohlfeld, K. Wehrle, A quantitative analysis of the impact of arbitrary blockchain content on bitcoin, in: International Conference on Financial Cryptography and Data Security, Springer, 2018, pp. 420–438.
- [34] General Data Protection Regulation (GDPR) - Official Legal Text - gdpr-info, 22, 2023. <https://gdpr-info.eu/>
- [35] Personal Information Protection Law of the People - Republic of China, 22, 2023. <http://en.npc.gov.cn.cdurl.cn/2021-12/29/c.694559.htm>
- [36] S.K. Lo, X. Xu, Y.K. Chiam, Q. Lu, Evaluating suitability of applying blockchain, in: 22nd International Conference on Engineering of Complex Computer Systems (ICECCS), IEEE, 2017, pp. 158–161.
- [37] R. Browne, Crypto platform hit by \$600 million heist asks hacker to become its chief security advisor, 27-09-2025, 2021. <https://www.cncb.com/2021/08/17/poly-network-cryptocurrency-hack-latest.html>
- [38] E. Andesta, F. Faghieh, M. Fooladgar, Testing smart contracts gets smarter, in: 2020 10th International Conference on Computer and Knowledge Engineering (ICCKE), IEEE, 2020, pp. 405–412.
- [39] M. Krichen, M. Lahami, Q.A. Al-Haija, Formal methods for the verification of smart contracts: a review, in: 2022 15th International Conference on Security of Information and Networks (SIN), IEEE, 2022, pp. 01–08.
- [40] N.Z. Benisi, M. Aminian, B. Javadi, Blockchain-based decentralized storage networks: a survey, J. Netw. Comput. Appl. 162 (2020) 102656.
- [41] Blockchain.com | Charts - Blockchain Size (MB) - blockchain.com [Accessed 01-05-2025], <https://www.blockchain.com/explorer/charts/blocks-size>
- [42] H. Krawczyk, T. Rabin, Chameleon hashing and signatures, Cryptology ePrint Archive, 1998.
- [43] A. Shamir, How to share a secret, Commun. ACM 22 (11) (1979) 612–613.
- [44] G. Ateniese, B.D. Medeiros, On the key exposure problem in chameleon hashes, in: International Conference on Security in Communication Networks, Springer, 2004, pp. 165–179.
- [45] J. Camenisch, D. Derler, S. Krenn, H.C. Pöhls, K. Samelin, D. Slamanig, Chameleon-hashes with ephemeral trapdoors, Springer, 2017.
- [46] D. Derler, K. Samelin, D. Slamanig, C. Striecks, Fine-grained and controlled rewriting in blockchains: Chameleon-hashing gone attribute-based Cryptology ePrint Archive, 2019.
- [47] J. Bethencourt, A. Sahai, B. Waters, Ciphertext-policy attribute-based encryption, in: 2007 IEEE Symposium on Security and Privacy (SP'07).
- [48] K. Ashritha, M. Sindhu, K. Lakshmy, Redactable blockchain using enhanced chameleon hash function, in: 2019 5th International Conference on Advanced Computing & Communication Systems (ICACCS).
- [49] Y. Tian, N. Li, Y. Li, P. Szalachowski, J. Zhou, Policy-based chameleon hash for blockchain rewriting with black-box accountability, in: Annual Computer Security Applications Conference, 2020, pp. 813–828.
- [50] S. Xu, X. Huang, J. Yuan, Y. Li, R.H. Deng, Accountable and fine-grained controllable rewriting in blockchains, IEEE Trans. Inf. Forensics Secur. 18 (2022) 101–116.
- [51] Y. Jia, S.-F. Sun, Y. Zhang, Z. Liu, D. Gu, Redactable blockchain supporting supervision and self-management, in: Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security, the 2021 ACM Asia Conference on Computer and Communications Security, 2021.
- [52] J. Ma, S. Xu, J. Ning, X. Huang, R.H. Deng, Redactable blockchain in decentralized setting, IEEE Trans. Inf. Forensics Secur. 17 (2022) 1227–1242.
- [53] W. Wang, J. Duan, L. Wang, H. Peng, L. Zhu, L. Li, Redactable blockchain supporting rewriting authorization without trapdoor exposure, in: IEEE Transactions on Dependable and Secure Computing 2025.
- [54] W. Wang, H. Peng, J. Duan, L. Wang, X. Hu, Z. Zhao, Resilient and redactable blockchain with two-level rewriting and version detection, IEEE Trans. Inf. Forensics Secur. 20 (2025) 1163–1175.
- [55] G. Zhou, X. Ding, H. Han, A. Zhu, Fine-grained redactable blockchain using trapdoor-hash, IEEE Internet Things J. 10 (24) (2023) 21203–21216.
- [56] K. Huang, X. Zhang, Y. Mu, X. Wang, G. Yang, X. Du, F. Rezaeibagha, Q. Xia, M. Guizani, Building redactable consortium blockchain for industrial internet-of-things, IEEE Trans. Ind. Inf. 15 (6) (2019) 3670–3679.
- [57] J. Shen, X. Chen, Z. Liu, W. Susilo, Verifiable and redactable blockchains with fully editing operations, IEEE Trans. Inf. Forensics Secur. 18 (2023) 3787–3802.
- [58] G. Panwar, R. Vishwanathan, S. Misra, ReTRACe: revocable and traceable blockchain rewrites using attribute-based cryptosystems, in: Proceedings of the 26th ACM Symposium on Access Control Models and Technologies, the 26th ACM Symposium on Access Control Models and Technologies, 2021, pp. 103–114.
- [59] X. Chen, Y. Gao, CDEDit: a highly applicable redactable blockchain with controllable editing privilege and diversified editing types, 2022. arXiv:2205.07054
- [60] C. Wu, L. Ke, Y. Du, Quantum resistant key-exposure free chameleon hash and applications in redactable blockchain, Inf. Sci. 548 (2021) 438–449.
- [61] K. Huang, X. Zhang, Y. Mu, F. Rezaeibagha, X. Du, N. Guizani, Achieving intelligent trust-layer for internet-of-things via self-redactable blockchain, IEEE Trans. Ind. Inf. 16 (2019) 2677–2686.
- [62] K. Huang, X. Zhang, Y. Mu, F. Rezaeibagha, X. Du, Scalable and redactable blockchain with update and anonymity, Inf. Sci. 546 (2021) 25–41.
- [63] Y. Tian, B. Liu, Y. Li, P. Szalachowski, J. Zhou, Accountable fine-grained blockchain rewriting in the permissionless setting, IEEE Trans. Inf. Forensics Secur. 19 (2023) 1756–1766.
- [64] D. Zhang, J. Le, X. Lei, T. Xiang, X. Liao, Secure redactable blockchain with dynamic support, IEEE Trans. Dependable Secure Comput. 21 (2) (2023) 717–731.
- [65] J. Li, H. Ma, J. Wang, Z. Song, W. Xu, R. Zhang, Wolverine: a scalable and transaction-consistent redactable permissionless blockchain, IEEE Trans. Inf. Forensics Secur. 18 (2023) 1653–1666.
- [66] R. Matzutt, V. Ahlrichs, J. Pennekamp, R. Karwacik, K. Wehrle, A moderation framework for the swift and transparent removal of illicit blockchain content, in: 2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), IEEE, 2022, pp. 1–9.
- [67] S. Agrawal, M. Chase, Fame: fast attribute-based message encryption, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 665–682.
- [68] V. Shoup, Practical threshold signatures, in: International Conference on the Theory and Applications of Cryptographic Techniques, Springer, 2000, pp. 207–220.
- [69] D. Galindo, J. Liu, M. Ordean, J.-M. Wong, Fully distributed verifiable random functions and their application to decentralised random beacons, in: 2021 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2021, pp. 88–102.
- [70] W. Wang, J. Duan, L. Wang, X. Hu, H. Peng, Strongly synchronized redactable blockchain based on verifiable delay functions, IEEE Internet Things J. 10 (19) (2023) 16778–16792.
- [71] W. Wang, L. Wang, J. Duan, X. Tong, H. Peng, Redactable blockchain based on decentralized trapdoor verifiable delay functions, in: IEEE Transactions on Information Forensics and Security 2024.
- [72] D. Grigoriev, V. Shpilrain, Rsa and redactable blockchains, Int. J. Comput. Math. Comput. Syst. Theory 6 (1) (2021) 1–6.
- [73] M.S. Dousti, A. Küpcü, Moderated redactable blockchains: a definitional framework with an efficient construct, in: Data Privacy Management, Cryptocurrencies and Blockchain Technology, Springer, 2020.
- [74] L. Cheng, J. Liu, C. Su, K. Liang, G. Xu, W. Wang, Polynomial-based modifiable blockchain structure for removing fraud transactions, Future Gener. Comput. Syst. 99 (2019) 154–163.
- [75] V. Botta, V. Iovino, I. Visconti, Towards data redaction in bitcoin, IEEE Trans. Netw. Serv. Manage. 19 (4) (2022) 3872–3883.
- [76] E. Larraia, M.S. Kiraz, O. Vaughan, How to redact the bitcoin backbone protocol, in: 2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), IEEE, 2024, pp. 485–493.
- [77] G. Avitabile, V. Botta, D. Friolo, I. Visconti, et al, Data redaction in smart-contract-enabled permissioned blockchains, 3791, CEUR-WS, 2024.
- [78] I. Puddu, A. Dmitrienko, S. Capkun, μ chain: how to forget without hard forks, Cryptology ePrint Archive, 2017.
- [79] J. Baron, K.E. Defrawy, J. Lampkins, R. Ostrovsky, Communication-optimal proactive secret sharing for dynamic groups, in: Applied Cryptography and Network Security: 13th International Conference, ACNS 2015, New York, NY, USA, Springer, 2015, pp. 23–41. Revised Selected Papers.
- [80] A. Dorri, S.S. Kanhere, R. Jurdak, MOF-BC: a memory optimized and flexible blockchain for large scale networks, Future Gener. Comput. Syst. 92 (2019) 357–373.
- [81] D. Deuber, B. Magri, S.A.K. Thyagarajan, Redactable blockchain in the permissionless setting, in: 2019 IEEE Symposium on Security and Privacy (SP), IEEE, 2019, pp. 124–138.
- [82] A. Marsalek, T. Zefferer, A correctable public blockchain, in: 2019 18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering, IEEE, 2019. TrustCom/BigDataSE.
- [83] J. Xu, X. Li, L. Yin, B. Guo, H. Feng, Z. Zhang, Redactable proof-of-stake blockchain with fast confirmation, Cryptology ePrint Archive, 2019.
- [84] X.-Y. Li, J. Xu, L.-Y. Yin, Y. Lu, Q. Tang, Z.-F. Zhang, Escaping from consensus: instantly redactable blockchain protocols in permissionless setting, in: IEEE Transactions on Dependable and Secure Computing 2022.
- [85] S.A.K. Thyagarajan, A. Bhat, B. Magri, D. Tschudi, A. Kate, in: Financial Cryptography and Data Security: 25th International Conference, FC 2021, Virtual Event, Springer, 2021. Reparo: Publicly verifiable layer to repair blockchains. Part II.
- [86] B. Luo, C. Yang, AeRChain: an anonymous and efficient redactable blockchain scheme based on proof-of-work, Entropy 25 (2) (2023) 270.
- [87] X. Meng, P. Ni, H. Li, H. Xu, A conflict-free redactable blockchain, in: 2022 IEEE International Conference on Pervasive Computing and Communications Workshops and Other Affiliated Events (PerCom Workshops), IEEE, 2022, pp. 285–290.
- [88] J. Duan, L. Gu, W. Wang, L. Wang, Chainknot, Redactable blockchain protocol with forced synchronization, IEEE Trans. Netw. Sci. Eng. 11 (3) (2024) 3091–3104.
- [89] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, N. Zeldovich, Algorand: scaling byzantine agreements for cryptocurrencies, in: Proceedings of the 26th Symposium on Operating Systems Principles, the 26th symposium on operating systems principles, 2017, pp. 51–68.
- [90] K.M. Alonso, et al., Zero to monero, 2020.
- [91] W. Dai, J. Liu, Y. Zhou, K.-K.R. Choo, X. Xie, D. Zou, H. Jin, PRBFT: a practical redactable blockchain framework with a public trapdoor, IEEE Trans. Inf. Forensics Secur. 19 (2024) 2425–2437.
- [92] I.E. Abid, Y. Benkaouz, A comparative analysis of blockchain redaction techniques, in: 2024 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS), IEEE, 2024, pp. 93–102.